

Chương 2: CÁC THUẬT TOÁN XỬ LÝ ẢNH (PHẦN 01)

Nội dung

1. Tổng quan về các thuật toán xử lý ảnh	2
2. Số hoá ảnh.....	3
2.1. Số hóa ảnh là gì	3
2.2. Mức độ lượng tử hóa	4
2.3. Nhận biết bước xám	5
3. Toán tử điểm ảnh (Point Operators).....	5
3.1. Nâng cao chất lượng ảnh	5
3.2. Toán tử điểm ảnh	6
3.3. Biến đổi Gamma (γ).....	8
3.4. Correct using pixel transform	11
3.5. Image Negative (Ảnh âm bản)	13
3.6. Combine image (Kỹ thuật kết hợp hình ảnh).....	13
3.7. Phép trừ ảnh (Absolute difference)	15
3.8. Trừ nền trong video (Video Background Subtraction)	17
4. Toán tử láng giềng	19
5. Histogram	20
6. Spatial Filtering - Các phép lọc trong không gian điểm ảnh	25
6.1. Lọc trung bình (Mean/Average Filter).....	28
6.2. Lọc Gaussian	32
6.3. Lọc Laplacian	34
6.4. Một số kỹ thuật lọc khác.....	36
7. Gradient and edge.....	37
8. Phụ lục	38
8.1. "Multimedia + Deep Learning" chắc chắn có sai số?	38
8.2. Cấu trúc ảnh: Foreground (Tiền cảnh) vs Background (Hậu cảnh).....	39
8.3. Bài toán FaceID: False Positive	39
8.4. Bài toán Phát hiện cạnh & kNN	40

1. Tổng quan về các thuật toán xử lý ảnh

- Quá trình xử lý ảnh được xem như là quá trình **thao tác ảnh đầu vào** nhằm cho ra kết quả mong muốn. Kết quả đầu ra của một quá trình xử lý ảnh **có thể là một ảnh “tốt hơn” hoặc một kết luận**.
- **Xử lý ảnh** tập trung vào việc thao tác và phân tích hình ảnh kỹ thuật số. Các thuật toán được phát triển để thực hiện các tác vụ khác nhau, từ những thao tác đơn giản như điều chỉnh độ sáng đến những nhiệm vụ phức tạp như nhận diện khuôn mặt.
- Một số nhóm thuật toán xử lý ảnh phổ biến:
 - **Các thuật toán cơ bản**
 - Đọc và hiển thị ảnh
 - Chuyển đổi không gian màu
 - Cắt xén và thay đổi kích thước
 - Điều chỉnh độ sáng, độ tương phản
 - **Lọc ảnh**
 - Lọc Linear:
 - Lọc trung bình: Làm mờ hình ảnh để giảm nhiễu (noise) → mỗi pixel được convert về pixel có giá trị trung bình.
 - ⇒ pros: nhanh, xử lý nhiễu chung
 - ⇒ cons: làm mờ nhiều, dễ mất chi tiết.
 - Lọc Gaussian: Làm mờ hình ảnh một cách trơn tru hơn → **pixel gần tâm có trọng số cao hơn** → làm mờ tự nhiên hơn.
 - ⇒ pros: mờ mượt hơn, vẫn keep được details, nhanh
 - ⇒ cons: làm mất chi tiết “nếu kernel lớn”
 - Lọc Non-Linear:
 - Lọc median: Loại bỏ các điểm nhiễu đơn lẻ → Thay mỗi pixel bằng **giá trị trung vị (median)** của các pixel xung quanh.
 - ⇒ pros: bỏ nhiễu tốt hơn Gaussian - xử lý nhiễu **Salt & Pepper (nhiều trắng đen lốm đốm)**
 - ⇒ cons: ảnh quá nhiễu/ kernel lớn → lọc sẽ không thể xử lý
 - Lọc bilateral: Kết hợp giữa lọc không gian (giống Gaussian) và lọc miền cường độ (intensity).
 - **Giữ cạnh (edges) rõ nét nhất**
 - Làm mờ và vẫn giữ được các đường biên (edge).
 - ⇒ pros: Blur được noise nhưng vẫn giữ chi tiết quan trọng.
 - ⇒ cons: chậm
 - **Phát hiện cạnh (edge detection)**
 - Toán tử Sobel: Phát hiện các cạnh theo cả phương ngang và dọc.
 - Toán tử Canny: Phát hiện cạnh với độ chính xác cao.
 - Laplacian: Phát hiện các cạnh dựa trên độ thay đổi đột ngột của cường độ pixel.

- **Phân đoạn ảnh (image segmentation)**
 - Phân ngưỡng (thresholding): Chia hình ảnh thành vùng dựa trên một **ngưỡng cường độ**.
 - K-means (Clustering): Phân cụm các pixel thành K nhóm. Chia toàn bộ ảnh thành **K nhóm** dựa trên **màu sắc hoặc cường độ sáng**. **Các pixel nào chung thì gom lại cụm**.
 - Otsu's thresholding: **Tự động tìm ngưỡng tốt nhất để phân chia ảnh thành hai vùng**, sao cho có sự khác biệt giữa 2 nhóm pixel là lớn nhất → thường dùng trong **"binary classification"**, Bài toán OCR (optical character recognition)
 - Region-based segmentation: Phân đoạn dựa trên các vùng liên thông có **cùng tính chất**.
- **Biến đổi hình học**
 - Phóng to, thu nhỏ: Thay đổi kích thước hình ảnh.
 - Xoay, lật: Thay đổi hướng của hình ảnh (đổi hàng & cột).
 - Biến đổi phối cảnh: Tạo hiệu ứng 3D.
- **Trích xuất đặc trưng**
 - SIFT - Scale-Invariant Feature Transform (chạy trên CPU)
 - SURF: Speeded Up Robust Features (giống SIFT nhưng nhanh hơn)
 - HOG: Histogram of Oriented Gradients
- **Nhận dạng vật thể**
 - Haar cascades: Phát hiện các đối tượng đơn giản như khuôn mặt, mắt.
 - Deep learning: Sử dụng mạng thần kinh sâu để nhận dạng các đối tượng phức tạp.

2. Số hoá ảnh

2.1. Số hóa ảnh là gì

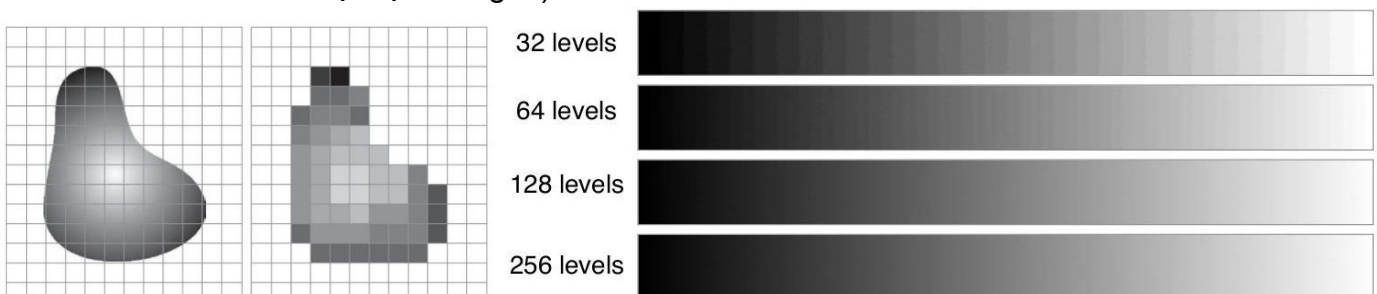
- Là **biến đổi một ảnh** (hay một hàm) **liên tục** trong không gian & theo giá trị, thành **dạng số rời rạc**
 - **"Liên tục" là gì? (Ảnh ngoài đời thực):** Hãy tưởng tượng bạn đang nhìn qua cửa sổ.
 - **Về không gian:** Cảnh vật bên ngoài liền mạch, không bị chia cắt. Bạn **không thấy "ô vuông"** nào cả.
 - **Về giá trị (màu sắc/độ sáng):** Ánh sáng **chuyển từ tối sang sáng** cực kỳ mượt mà. Có **vô số mức độ sáng** khác nhau (vô tận), không đếm được.
→ Đây là **Ảnh liên tục (Analog)**. Máy tính không thể lưu trữ cái "vô tận" này được.
 - **"Rời rạc" là gì? (Ảnh trên máy tính):** Để đưa cảnh vật đó vào máy tính, ta phải **"số hóa"** nó bằng cách **áp một cái lưới ô vuông** lên cửa sổ. **Quá trình số hóa ảnh gồm 2 bước:**
 - **Bước 1: Rời rạc hóa về KHÔNG GIAN (Sampling - Lấy mẫu):**
 - *Đo giá trị trong các khoảng không gian*
 - Thay vì lấy toàn bộ cảnh, ta **chia bức tranh thành hàng nghìn ô vuông nhỏ (chính là Pixel)**. Mỗi ô vuông này có một tọa độ xác định (Hàng 1, Cột 1).
→ Từ một bức tranh liền mạch, nó thành một tập hợp các ô vuông rời rạc.

▪ **Bước 2: Rời rạc hóa về GIÁ TRỊ (Quantization - Lượng tử hóa):**

- Ánh xạ cường độ (hoặc giá trị) đo được thành một số hữu hạn các mức rời rạc
- Máy tính không thể lưu "màu xanh hơi nhạt một tí tẹo". Nó cần một con số chính xác. Nó sẽ ép màu sắc của từng ô vuông về một con số cụ thể trong thang đo có sẵn (ví dụ: từ 0 đến 255).
→ Từ vô vàn mức độ sáng, giờ ta chỉ còn 256 mức độ cố định (rời rạc).

2.2. Mức độ lượng tử hóa

- **Số lượng mức xám** được yêu cầu trên một bức ảnh số tương ứng với **số lượng bits** trên mỗi pixel.
 - Hãy coi **Bits** chính là **kích thước hộp bút chì** của bạn.
 - **1 Bit (Hộp 2 màu):** Bạn chỉ có 2 cây bút: Đen tuyền (0) và Trắng tinh (1). Không có màu xám. Ảnh tô ra sẽ rất gắt (chỉ có mảng đen hoặc trắng).
 - **1 bit** → Tương ứng có 2 trường hợp: hoặc đen, hoặc trắng.
 - **2 Bits (Hộp 4 màu):** Bạn có thêm 2 cây màu xám nữa (Xám đậm, Xám nhạt). Ảnh bắt đầu dịu hơn một chút.
 - **8 Bits (Hộp 256 màu):** Bạn có một hộp bút khổng lồ với **256 cây bút** chì trải dài từ Đen → Xám đậm → Xám vừa → Xám nhạt → Trắng.
 - **Số bit càng cao (hộp bút càng to)** → **Bạn càng có nhiều sắc độ để lựa chọn** → **Bức ảnh càng mượt mà, giống thật.** (Số lượng mức xám càng cao thì ảnh nhìn càng "mượt").
 - **Mức độ lượng tử hóa càng cao thì ảnh càng mịn, tổn bộ nhớ càng nhiều** → **Sự thách thức khi cân bằng chi phí & công nghệ.**
- **Ảnh số thường được lượng tử hoá thành 256 mức xám.**
 - Con số 256 không phải ngẫu nhiên, nó đến từ 2 lý do:
 - **Lý do kỹ thuật (Máy tính):**
 - Máy tính dùng hệ nhị phân. 1 đơn vị lưu trữ cơ bản nhất là 1 Byte = 8 bits.
 - **Công thức tính số màu là 2^n (với n là số bit).**
 - $2^8 = 256$ mức giá trị (từ 0 đến 255). Vì vậy, dùng 8-bit (256 mức) là con số vừa vặn, chuẩn chỉnh nhất với phần cứng máy tính.
 - **Lý do sinh học (Mắt người):** Mắt con người nhìn thấy sự chuyển đổi ánh sáng. Với 256 mức độ từ đen sang trắng, **các bước chuyển màu đã đủ nhỏ để mắt ta không còn phân biệt được ranh giới giữa các vệt màu nữa.** Ta sẽ thấy nó mượt như một dải lụa. (Nếu dùng ít hơn, ví dụ 16 mức, bạn sẽ thấy ảnh bị chia thành các vệt sọc loang lổ).



2.3. Nhận biết bước xám

- Khả năng của các thuật toán và phần mềm phân tích hình ảnh trong việc **phân biệt và đo lường các mức độ xám** khác nhau trong một bức ảnh.
- Khả năng này đóng vai trò quan trọng trong nhiều ứng dụng xử lý ảnh, như:
 - **Cân bằng sáng tối:** Điều chỉnh các mức độ sáng tối để tạo ra hình ảnh hài hòa và tự nhiên.
 - **Phân đoạn ảnh:** Tách các đối tượng khỏi nền dựa trên sự khác biệt về mức độ xám.
 - **Nhận diện vật thể:** Tìm kiếm và xác định các vật thể trong ảnh dựa trên các đặc trưng về màu sắc và độ sáng.
 - **Xử lý ảnh:** Các hiệu ứng như tăng cường độ tương phản, làm mờ, hoặc tạo các hiệu ứng nghệ thuật thường dựa trên việc điều chỉnh các mức độ xám.
- **Nếu mức xám dùng 8 bit nó sẽ là dải liên tục.**

Dải liên tục



Mã hoá tuyến tính dùng 32 mức (5 bits)



Nhiều mức xám không khác nhau khi nhìn bằng mắt thường

3. Toán tử điểm ảnh (Point Operators)

3.1. Nâng cao chất lượng ảnh

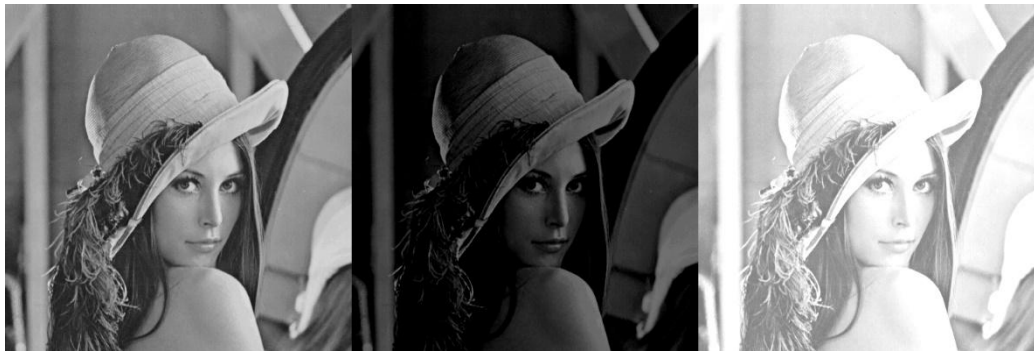
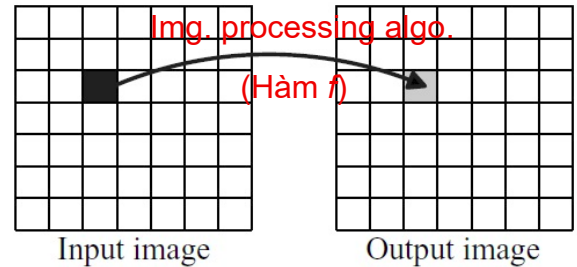
- Nâng cao chất lượng là bước cần thiết trong xử lý ảnh nhằm *hoàn thiện một số đặc tính của ảnh*. Nâng cao chất lượng ảnh gồm hai công đoạn khác nhau: **Tăng cường ảnh và khôi phục ảnh**.
- **Tăng cường ảnh** nhằm hoàn thiện các đặc tính của ảnh như:
 - Lọc nhiễu, hay làm trơn ảnh.
 - Tăng độ tương phản, điều chỉnh mức xám của ảnh.
 - Làm nổi biên ảnh.
- **Khôi phục ảnh:** Nhằm khôi phục ảnh gần với trạng thái thực nhất trước khi biến dạng, tùy theo nguyên nhân gây ra biến dạng.
- **Các phương pháp thực hiện:**
 - **Thực hiện trên miền không gian**
 - **Toán tử điểm (Point Operations):** Giá trị 1 điểm ảnh đầu ra phụ thuộc duy nhất vào 1 giá trị đầu vào tại vị trí tương ứng trên ảnh vào.
 - **Toán tử cục bộ (Local Operations):** Giá trị một điểm ảnh đầu ra phụ thuộc vào giá trị của chính nó và các lân cận của nó trong ảnh vào.

- **Thực hiện trên miền tần số**

- Toán tử toàn cục (Global Operations): Giá trị của 1 điểm ảnh đầu ra phụ thuộc vào tất cả giá trị các điểm ảnh trong ảnh vào.

3.2. Toán tử điểm ảnh

- Là một phép biến đổi ảnh số bằng cách **áp dụng một hàm toán học lên từng điểm ảnh riêng lẻ** của ảnh. Giá trị mới của mỗi điểm ảnh sẽ **chỉ phụ thuộc vào giá trị gốc** của điểm ảnh đó.
- A point operator, denoted as Ψ , transforms an *input image f* into an *output image g* where each pixel's value in *g* is determined by a function ψ applied to the corresponding pixel in *f*. Mathematically, this can be expressed as: **$g(x) = \psi(f(x))$**
- **Cách thức hoạt động**
 - Mỗi điểm ảnh là một số, mỗi số tương ứng với giá trị màu của một điểm ảnh.
 - Áp dụng hàm toán học lên **giá trị của mỗi điểm ảnh để tính ra giá trị mới**. Hàm này có thể đơn giản như cộng, trừ, nhân, chia hoặc phức tạp hơn như hàm logarit, hàm mũ.
 - Tạo ảnh mới thông qua các giá trị mới của các điểm ảnh.
- **Các toán tử điểm ảnh**
 - **Điều chỉnh độ sáng**: Tăng hoặc giảm giá trị của **mỗi điểm ảnh** để làm cho ảnh sáng hơn hoặc tối hơn.
 - **Công thức**:
 - Tăng độ sáng: **pixel_mới = pixel_cũ + giá_trị_tăng**
 - Giảm độ sáng: **pixel_mới = pixel_cũ - giá_trị_giảm**
 - **Ví dụ**: Giả sử ta có một ảnh xám với các giá trị điểm ảnh từ 0 đến 255. Để tăng độ sáng của ảnh lên 50 đơn vị, ta **cộng thêm 50 vào giá trị của mỗi điểm ảnh**.



- **Phân ngưỡng**: Chuyển đổi **ảnh xám thành ảnh nhị phân (đen trắng)** bằng cách so sánh giá trị của mỗi điểm ảnh với một ngưỡng nhất định.
 - Image thresholding simplifies images by converting them into a binary format (black and white), making it *easier to identify objects, extract features, and perform other image processing tasks*.

▪ **Công thức:**

- Nếu $\text{pixel_cũ} \geq \text{ngưỡng}$: $\text{pixel_mới} = 255$ (màu trắng)
- Nếu $\text{pixel_cũ} < \text{ngưỡng}$: $\text{pixel_mới} = 0$ (màu đen)

- **Ví dụ:** Chọn ngưỡng là 128. Các điểm ảnh có giá trị lớn hơn hoặc bằng 128 sẽ trở thành màu trắng, còn lại là màu đen.



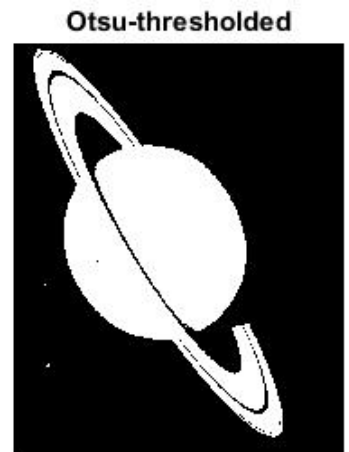
Original Image



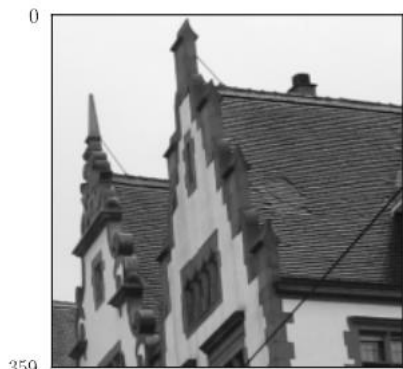
Thresholded Image



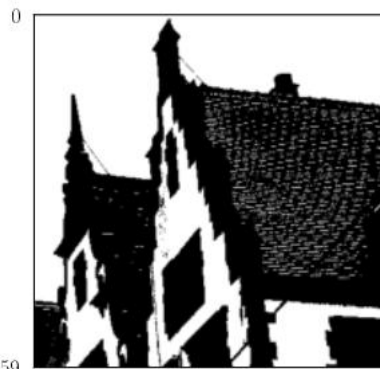
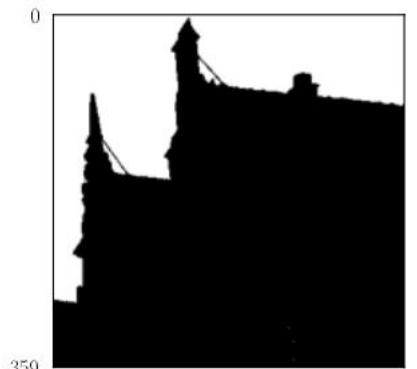
Original



Otsu-thresholded



Original image

 $T = 150$  $T = 200$

- **Đảo ngược âm bản:** Đảo ngược màu sắc của mỗi điểm ảnh.

- **Công thức:** $\text{pixel_mới} = 255 - \text{pixel_cũ}$

- **Ví dụ:** Nếu một điểm ảnh có giá trị là 50, sau khi đảo ngược sẽ có giá trị là $255 - 50 = 205$.

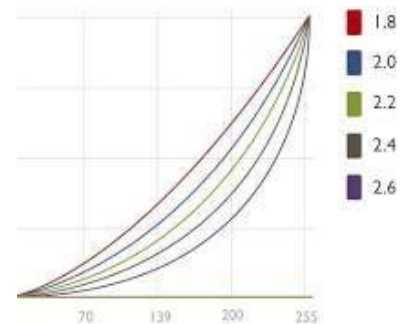
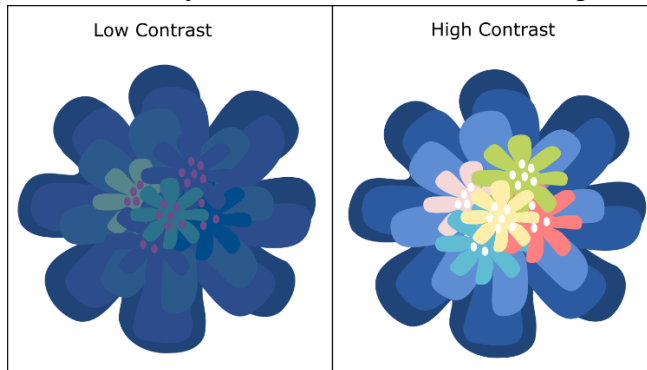


- **Cải thiện độ tương phản:** Tăng cường sự khác biệt giữa các vùng sáng và tối trong ảnh.
 - **Công thức:** $\text{pixel_mới} = \text{Hàm_Toán_Học}(\text{pixel_cũ})$
 - Có nhiều hàm toán học khác nhau để cải thiện độ tương phản, chẳng hạn như: hàm gamma, hàm logarit, hàm mũ.
 - **Ví dụ:** Áp dụng **hàm gamma** với một giá trị gamma phù hợp để tăng độ tương phản của ảnh.

3.3. Biến đổi Gamma (γ)

- Trong ngữ cảnh xử lý hình ảnh, Gamma là một thuật ngữ dùng để mô tả **mối quan hệ phi tuyến giữa giá trị đầu vào (tín hiệu) và giá trị đầu ra (độ sáng)** của một thiết bị hiển thị như màn hình máy tính. Nói một cách đơn giản, **gamma quyết định cách các màu sắc sáng tối được hiển thị trên màn hình.**
- **Biến đổi Gamma** là một phép toán phi tuyến, có tác dụng **điều chỉnh độ tương phản của một hình ảnh** bằng cách **thay đổi mối quan hệ giữa giá trị đầu vào (pixel gốc) và giá trị đầu ra (pixel sau khi biến đổi).**
- **Tại sao cần biến đổi Gamma?**
 - **Hiệu chỉnh độ sáng:** Mỗi thiết bị hiển thị (màn hình, máy in) có đặc tính gamma khác nhau. Biến đổi gamma giúp điều chỉnh **để hình ảnh hiển thị đúng với ý định** của người tạo.
 - **Tăng cường độ tương phản:** Đối với các hình ảnh có độ tương phản thấp, biến đổi gamma có thể giúp làm **nổi bật các chi tiết sáng tối.**
 - **Điều chỉnh màu sắc:** Mặc dù chủ yếu ảnh hưởng đến độ sáng, biến đổi gamma cũng có thể **gián tiếp ảnh hưởng đến màu sắc** của hình ảnh.
- **Ứng dụng của biến đổi Gamma**
 - Hiệu chỉnh màu sắc: Đảm bảo hình ảnh hiển thị đúng **trên các thiết bị** khác nhau.
 - Xử lý ảnh: Tăng cường độ tương phản, điều chỉnh độ sáng, tạo các hiệu ứng đặc biệt.
 - Đồ họa máy tính: Tạo ra các hiệu ứng ánh sáng, bóng đổ, và các hiệu ứng hình ảnh khác.
 - Xử lý video: Cân bằng màu sắc và độ sáng trong video.
- **Sử dụng Gamma để điều chỉnh độ tương phản**
 - **Công thức biến đổi Gamma** có dạng: $V_{\text{out}} = V_{\text{in}}^{\text{gamma}}$ trong đó:
 - V_{in} : Giá trị pixel gốc (trong khoảng từ 0 đến 1 → Phải chuẩn hóa (normalize) nếu sử dụng các hệ màu khác)
 - V_{out} : Giá trị pixel sau khi biến đổi
 - gamma: Tham số gamma, quyết định mức độ biến đổi.
 - Gamma < 1: **Tăng độ tương phản ở vùng sáng**, làm **tối các vùng tối.**
 - Gamma > 1: **Tăng độ tương phản ở vùng tối**, làm **sáng các vùng sáng.**
 - Gamma = 1: Không có thay đổi nào.
 - Dùng phép nhân (hàm mũ) để tăng cường sự thay đổi (thay đổi nhiều, nhanh hơn), thay vì phép cộng/trừ làm quá trình chuyển đổi diễn ra từ từ, chậm hơn.

- **Hình ảnh có độ tương phản cao** là những bức ảnh có sự khác biệt rõ rệt giữa vùng sáng nhất và vùng tối nhất.
 - Độ tương phản cao thường làm **tăng độ bão hòa** của màu sắc, khiến chúng trở nên sống động và nổi bật hơn.
 - **Độ bão hòa (Saturation)** hiểu đơn giản là **độ tinh khiết** hoặc **độ rực rỡ** của một màu sắc. Nó đo lường xem màu đó "đậm đà" đến mức nào hay "nhợt nhạt" đến mức nào.
 - Độ bão hòa CAO (High Saturation) = Màu nguyên chất được bóp thẳng từ tuýp nước màu ra, không pha trộn gì cả.
 - Độ bão hòa THẤP (Low Saturation) = Bạn lấy màu nguyên chất pha thêm nhiều **nước** hoặc pha thêm màu **xám/trắng**.
 - **Độ bão hòa bằng 0 (Desaturated)** = Khi bạn rút hết độ bão hòa đi, màu sắc biến mất hoàn toàn. Bức ảnh trở thành Grayscale.
 - Các chi tiết trong vùng sáng và tối đều được làm nổi bật, giúp người xem dễ dàng nhận biết và phân biệt chúng
- **Hình ảnh có độ tương phản thấp**, khiến cho các chi tiết khó nhìn rõ.
 - Màu sắc nhạt nhòa, **không có sự tương phản rõ ràng giữa các vùng sáng và tối**.
 - Hình ảnh này có thể được cải thiện bằng cách tăng độ tương phản.



- **Đường cong gamma** biểu diễn **mối quan hệ giữa giá trị đầu vào và giá trị đầu ra**.
 - Đường cong này có thể là **tuyến tính** (gamma = 1) hoặc **phi tuyến** (gamma khác 1).
 - **Gamma thấp**: Đường cong gamma sẽ dẹt hơn, làm **giảm độ tương phản**.
 - **Gamma cao**: Đường cong gamma sẽ cong hơn, **tăng độ tương phản**.

- Ví dụ xây dựng hàm **adjust_image_gamma** với thư viện cv2 và PIL

	Cách 1: NumPy (OpenCV style)	Cách 2: PIL (Pillow style)
	<pre>import cv2 def adjust_image_gamma(image, gamma): """Adjusts the gamma of an image. Args: image: The input image as a NumPy array. gamma: The gamma value to apply. Returns: The gamma-adjusted image. """ # Convert the image to floating-point data type for calculations image = image.astype(float) / 255.0 # Apply the gamma correction formula adjusted_image = (image ** gamma) * 255 # Convert the image back to integer data type for display adjusted_image = adjusted_image.astype(int) return adjusted_image</pre>	<pre>from PIL import Image def adjust_image_gamma_pil(image, gamma): """Adjusts the gamma of an image using PIL. Args: image: The input PIL Image object. gamma: The gamma value to apply. Returns: The gamma-adjusted PIL Image object. """ # Convert the image to floating-point data type for calculations image = image.convert('F') # Apply the gamma correction formula adjusted_image = image ** gamma # Convert the image back to integer data type for display adjusted_image = adjusted_image.convert('RGB') return adjusted_image</pre>
	cv2 thích hợp với nhiều ngôn ngữ. Dùng khi Đầu vào là Video, hay với Deep Learning, & Cần tốc độ cực cao	PIL chỉ dùng với Python, Highly recommend (cách dùng PIL an toàn, ngắn gọn và ít lỗi vặt hơn)
Độ phức tạp	Cao hơn. Bạn phải tự tay chuẩn hóa (chia 255), tính toán, rồi nhân lại, rồi ép kiểu (casting).	Thấp hơn. Code ngắn gọn, sử dụng các phương thức có sẵn như <code>.convert()</code> .
Quản lý bộ nhớ	Tạo ra các mảng tạm thời (temporary arrays) trong quá trình tính toán vector.	PIL tối ưu hóa quản lý bộ nhớ bên dưới lớp vỏ bọc (wrapper).
Xử lý lỗi	Rủi ro. Nếu tính toán ra số > 255 hoặc < 0, bạn phải tự dùng np.clip trước khi ép kiểu, nếu không sẽ bị lỗi màu. • Nếu phép tính lũy thừa vô tình tạo ra giá trị $255.5 \rightarrow$ nó thành 255 (OK). • Nhưng nếu gamma làm giá trị vọt lên 256 (do sai số làm tròn floating point) $\rightarrow$ <code>astype(uint8)</code> (thường dùng cho ảnh) sẽ biến nó thành 0 (màu đen xì). Đây là lỗi "tràn số" (overflow). <i>Đoạn code mẫu thiếu bước clip này.</i>	An toàn. Khi gọi <code>.convert('RGB')</code>, PIL tự động xử lý việc "kẹp" (clamp) giá trị về 0-255 một cách chuẩn xác. PIL chuyển sang mode 'F' (32-bit floating point pixels) giúp giữ độ chính xác rất cao trong lúc tính toán lũy thừa (gamma), sau đó mới nén lại về 8-bit. Điều này giảm thiểu hiện tượng "gãy màu" (banding) tốt hơn so với việc tính toán số học thông thường nếu không cẩn thận.
Hiệu năng	Rất nhanh với xử lý ma trận số lượng lớn (Batch processing).	Nhanh với xử lý ảnh đơn lẻ, nhưng có thể chậm hơn NumPy một chút ở quy mô lớn.

3.4. Correct using pixel transform

- **Thay đổi từng điểm ảnh** (pixel) trong một hình ảnh để đạt được hiệu ứng hoặc cải thiện chất lượng hình ảnh mong muốn.
- **Mục đích của việc điều chỉnh từng pixel:**
 - Nâng cao chất lượng hình ảnh: Tăng độ sáng, độ tương phản, cân bằng màu sắc, độ nét.
 - Khôi phục hình ảnh: Loại bỏ nhiễu, vết xước, biến dạng.
 - Tạo hiệu ứng: Làm mờ, làm sắc nét, phát hiện cạnh, và nhiều hiệu ứng khác.
- **Các phương pháp biến đổi pixel phổ biến:**
 - **Biến đổi tuyến tính**
 - **Độ sáng:** Cộng thêm một giá trị cố định vào mỗi pixel.
 - **Độ tương phản:** Nhân giá trị mỗi pixel với một hằng số.
 - **Ảnh âm bản:** Trừ giá trị mỗi pixel đi giá trị lớn nhất có thể.
 - **Biến đổi phi tuyến**
 - **Hiệu chỉnh Gamma:** Điều chỉnh độ sáng và độ tương phản tổng thể của hình ảnh bằng một hàm mũ.
 - **Cân bằng histogram:** *Kéo giãn histogram* của hình ảnh để cải thiện độ tương phản.
 - **Biến đổi logarit:** **Nén dải động** của hình ảnh, hữu ích cho hình ảnh có độ tương phản cao.
 - **Lọc không gian**
 - **Convolution:** **Áp dụng một kernel (ma trận) lên mỗi pixel** để tạo ra một giá trị pixel mới dựa trên các pixel lân cận.
 - **Làm mờ:** Làm mờ hình ảnh để giảm nhiễu và chi tiết.
 - **Làm sắc nét:** Nâng cao các cạnh và chi tiết.
 - **Phát hiện cạnh:** Xác định ranh giới giữa các vùng khác nhau trong hình ảnh.
- **Ứng dụng của biến đổi pixel:**
 - Hình ảnh y tế: Nâng cao hình ảnh để chẩn đoán và phân tích.
 - Viễn thám: Xử lý ảnh vệ tinh để phân loại sử dụng đất và theo dõi môi trường.
 - Tầm nhìn máy tính: Trích xuất thông tin từ hình ảnh cho các nhiệm vụ như nhận dạng vật thể và theo dõi.
 - Ảnh kỹ thuật số: Cải thiện chất lượng ảnh.

```
def pixel_transform(image, alpha = 1.0, beta = 0):
    ...
    out[pixel] = alpha * image[pixel] + beta
    ...

    output = np.zeros(image.shape, image.dtype)
    h, w, ch = image.shape
    for y in range(h):
        for x in range(w):
            for c in range(ch):
                output[y,x,c] = np.clip(alpha*image[y,x,c] + beta, 0, 255)

    return output
```

```
transformed_high = pixel_transform(high, 0.5, 20)
plt.imshow(transformed_high[:,::-1])
```

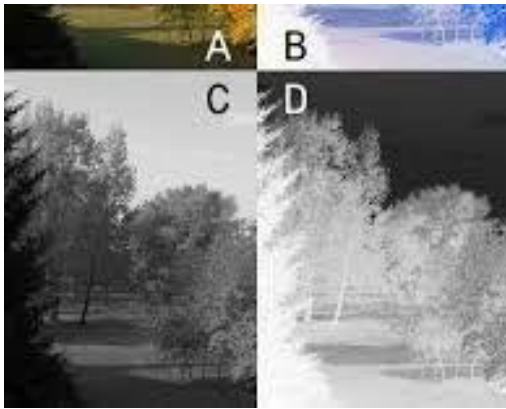
- Đoạn code dùng **Toán tử điểm (Point Operator)** thủ công để điều chỉnh **Độ tương phản (Contrast)** và **Độ sáng (Brightness)** của ảnh.
- **Mục đích của hàm pixel_transform:** Hàm này áp dụng công thức biến đổi tuyến tính lên từng điểm ảnh: $g(x, y) = \alpha \cdot f(x, y) + \beta$
 - α (**alpha**): Hệ số điều chỉnh **độ tương phản** (Gain), $\alpha > 1$: Tăng tương phản, $0 < \alpha < 1$: Giảm tương phản.
 - β (**beta**): Hệ số điều chỉnh **độ sáng** (Bias), $\beta > 0$: Làm ảnh sáng lên, $\beta < 0$: Làm ảnh tối đi.
- **Khởi tạo ảnh đầu ra:** `output = np.zeros(image.shape, image.dtype)`
Tạo một tấm ảnh đen (toàn số 0) có cùng kích thước (image.shape) và cùng kiểu dữ liệu (image.dtype - thường là uint8) với ảnh gốc để chứa kết quả.
- **3 Vòng lặp lồng nhau (Nested Loops):**

```
for y in range(h): # Duyệt từng dòng (chiều cao)
    for x in range(w): # Duyệt từng cột (chiều rộng)
        for c in range(ch): # Duyệt từng kênh màu (R, G, B)
```

 - Đây là cách xử lý thủ công nhất. Máy tính sẽ đi qua từng ô màu nhỏ xíu một.
 - *Lưu ý:* Cách viết này rất tốt để hiểu bản chất "Toán tử điểm" (xử lý từng điểm riêng biệt), nhưng trong thực tế Python chạy vòng lặp này **RẤT CHẬM**.
- **Công thức cốt lõi & Hàm np.clip:** `output[y,x,c] = np.clip(alpha*image[y,x,c] + beta, 0, 255)`
 - $\alpha * \text{image}[\dots] + \beta$: Tính toán **giá trị màu mới**.
 - `np.clip(..., 0, 255)`: **Cực kỳ quan trọng**.
 - Nếu tính ra > 255 (ví dụ 300) → Ép về 255 (Tránh lỗi tràn số/quay vòng về 44).
 - Nếu tính ra < 0 (ví dụ -20) → Ép về 0.
 - Nếu không có clip, ảnh sẽ bị nhiễu hạt lấm tấm rất xấu (noise).
- **Phản hiển thị (Display):** `plt.imshow(transformed_high[:,::-1])`
 - `transformed_high`: Là **ảnh kết quả sau khi chỉnh sửa**.
 - `[:,::-1]`: Đây là **kỹ thuật đảo ngược kênh màu (Slicing)**.
 - Khả năng cao ảnh đầu vào `high` được đọc bằng **OpenCV** (mặc định là chuẩn **BGR** - Xanh dương trước).
 - Thư viện **Matplotlib** (plt) lại hiển thị theo chuẩn **RGB** (Đỏ trước).
 - **Lệnh `::-1` giúp đảo ngược thứ tự B-G-R thành R-G-B** để ảnh hiện lên đúng màu, không bị ám xanh dương.

3.5. Image Negative (Ảnh âm bản)

- Các tông màu **sáng** trong hình ảnh gốc sẽ trở **thành tối và ngược lại**.
- Nói cách khác, *ảnh âm bản là một phiên bản "đảo ngược" của hình ảnh gốc*.



```
import cv2
import numpy as np

def image_negative(image):
    """Inverts the colors of an image.

    Args:
        image: The input image as a NumPy array.

    Returns:
        The inverted image as a NumPy array.
    """

    # Create a negative image by subtracting each pixel value from 255
    negative_image = 255 - image

    return negative_image

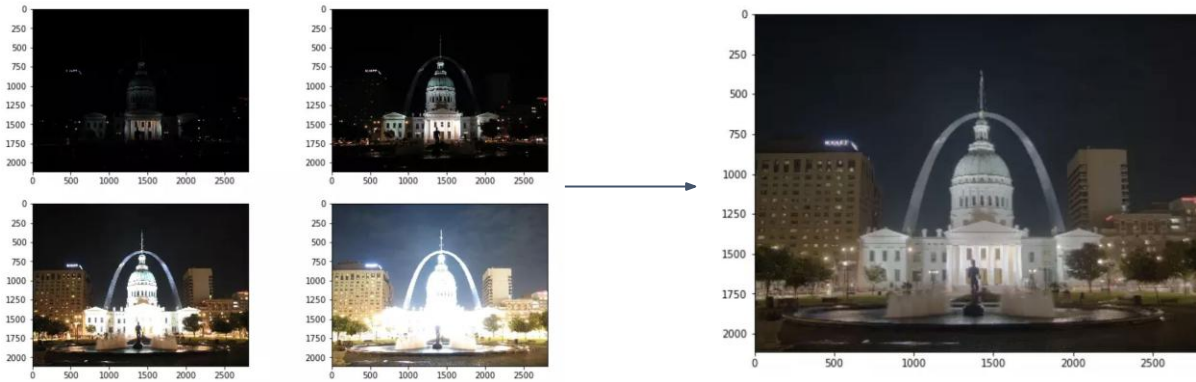
# Load an image
image = cv2.imread('image.jpg')

# Invert the colors of the image
negative_image = image_negative(image)

# Display the original and inverted images
cv2.imshow('Original', image)
cv2.imshow('Negative', negative_image)
cv2.waitKey(0)
```

3.6. Combine image (Kỹ thuật kết hợp hình ảnh)

- Hay còn gọi là **ghép ảnh, ghép ảnh**, là một phương pháp cho phép **kết hợp hai hoặc nhiều hình ảnh khác nhau để tạo ra một tác phẩm hoàn toàn mới**.
- **Các phương pháp kết hợp hình ảnh phổ biến:**
 - Overlay (trồng chồng): **Đặt một hình ảnh lên trên hình ảnh khác**, tạo ra hiệu ứng trong suốt hoặc mờ dần.
 - Masking (tạo mặt nạ): Sử dụng một lớp mặt nạ để **che đi một phần của hình ảnh và thay thế bằng một phần của hình ảnh khác**.
 - Blending (trộn màu): **Trộn màu sắc của hai hình ảnh** để tạo ra một hiệu ứng chuyển tiếp mượt mà.
 - **Stitching (ghép nối)**: Kết hợp nhiều hình ảnh lại với nhau để **tạo ra một bức ảnh toàn cảnh** → Đây là **Biến đổi hình học (Geometric Transformation)** kết hợp với **Toán tử lân cận**.
 - Morphing (biến hình): **Chuyển đổi dần dần từ hình ảnh này sang hình ảnh khác**.
- Nếu "Combine" là **chồng đè** lên nhau (Cộng, Trừ, Trộn) → **Toán tử điểm**.
- Nếu "Combine" là **nối dài** cạnh nhau (Panorama) → **Biến đổi hình học**.



```
import cv2

# Load the two images
image1 = cv2.imread("image1.jpg")
image2 = cv2.imread("image2.jpg")

# Resize the images to the same size
image1 = cv2.resize(image1, (500, 500))
image2 = cv2.resize(image2, (500, 500))

# Combine the images using the overlay method
combined_image = cv2.addWeighted(image1, 0.5, image2, 0.5, 0)

# Display the combined image
cv2.imshow("Combined image", combined_image)
cv2.waitKey(0)
```

- **Cộng ảnh / Trộn ảnh (Image Addition / Blending):** Máy tính lấy giá trị pixel của ảnh 1 nhân với một hệ số, cộng với pixel của ảnh 2 nhân hệ số, rồi ra kết quả.
- Hàm **cv2.addWeighted(image1, 0.5, image2, 0.5, 0)** → Công thức toán học bên dưới hàm này là:

$$Output(x, y) = [0.5 \times Image_1(x, y)] + [0.5 \times Image_2(x, y)] + 0$$

$$Output = (Image_1 \times \alpha) + (Image_2 \times \beta) + \gamma$$

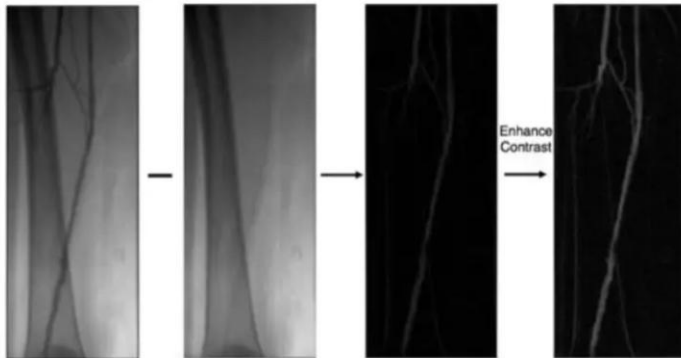
- **0.5 (α):** Trọng số của ảnh 1. Nghĩa là lấy **50% độ đậm** của ảnh này.
- **0.5 (β):** Trọng số của ảnh 2. Nghĩa là lấy **50% độ đậm** của ảnh này.
- **0 (γ):** Độ sáng bổ sung. Cộng thêm 0 vào kết quả (không làm sáng thêm hay tối đi).
- Để tính điểm ảnh tại tọa độ (x, y), máy tính chỉ cần nhìn đúng vào tọa độ (x, y) của hai ảnh thành phần. Nó **không** cần nhìn sang hàng xóm. → **Là Toán tử điểm.**
- **Đầu ra là một bức ảnh mới** có cùng kích thước và kiểu dữ liệu với hai ảnh gốc, trong đó:
 - Hình ảnh của cả image1 và image2 đều hiện lên cùng lúc.
 - Vì mỗi ảnh chỉ giữ 50% độ đậm (0.5), nên chúng trông **mờ ảo và lồng vào nhau**. Bạn sẽ nhìn xuyên qua ảnh này để thấy ảnh kia.
- Để hàm này hoạt động, **bắt buộc image1 và image2 phải có cùng kích thước (chiều cao, chiều rộng) và cùng số kênh màu**. Đó là lý do tại sao trong code, trước khi gọi hàm này, người ta phải có bước cv2.resize(image, (500, 500)) để ép hai ảnh về cùng cỡ 500x500.

- **Ứng dụng:** Bài toán Anti Face Morphing Attack (Chống tấn công biến đổi khuôn mặt) hoặc đơn giản là Anti Ghost Image Fraud.
 - Kỹ thuật "chồng ảnh" chính là **cơ chế cốt lõi** để tạo ra những khuôn mặt giả mạo này.
 - Kỹ thuật này hoạt động dựa trên việc trộn lẫn khuôn mặt của hai người khác nhau:
 - **Người A (Tội phạm/Kẻ tấn công):** Muốn trốn tránh sự truy nã hoặc muốn truy cập tài khoản của người khác.
 - **Người B (Nạn nhân/Đồng phạm sạch):** Người có lý lịch trong sạch, không có tiền án tiền sự.
 - Quy trình "Chồng ảnh":
 - Kẻ tấn công sẽ lấy ảnh của A và B, dùng thuật toán để căn chỉnh mắt, mũi, miệng cho khớp nhau, sau đó dùng phép cộng ảnh (Alpha Blending) để trộn lại.
 - → **Kết quả: Tạo ra khuôn mặt C.** Khuôn mặt C này nhìn thoáng qua thì giống B, nhưng các đặc điểm hình học (khoảng cách mắt, khung xương) lại chứa đựng thông tin của cả A.
 - Hệ thống nhận diện khuôn mặt (AI) không nhìn ảnh như mắt người. Nó chuyển ảnh thành các **vector số học** (một dãy số đại diện cho các đặc điểm). Khi ảnh C được tạo ra từ 50% A và 50% B:
 - **Đối với AI:** Vector của ảnh C nằm ở vị trí "trung gian" trong không gian toán học. Nó đủ gần với A để hệ thống nhận diện là A, và cũng đủ gần với B để hệ thống nhận diện là B.
 - **Hậu quả:**
 - Tội phạm dùng ảnh C để đăng ký làm thẻ căn cước/passport hoặc tài khoản ngân hàng (dưới danh nghĩa người B).
 - Hệ thống duyệt hồ sơ (hoặc nhân viên kiểm duyệt) nhìn vào thấy giống người B → **Duyệt**.
 - Sau khi có thẻ/tài khoản, **Tội phạm A** dùng khuôn mặt thật của hắn để đăng nhập hoặc đi qua cổng an ninh tự động.
 - Máy quét mặt A, so sánh với ảnh C trong dữ liệu. Vì C chứa 50% đặc điểm của A → Máy báo "**Trùng khớp**" → **Lọt lưới**.
 - Việc cộng 2 bức ảnh lại có thể qua mặt hệ thống bảo mật.

3.7. Phép trừ ảnh (Absolute difference)

- **Tìm ra những điểm khác biệt giữa hai hình ảnh.**
- Khi thực hiện phép trừ, **các pixel tương ứng trong hai hình ảnh sẽ được trừ cho nhau.**
- **Kết quả thu được sẽ là một hình ảnh mới,** trong đó **các điểm sáng thể hiện những vùng có sự khác biệt lớn** giữa hai hình ảnh gốc.
- **Mục đích sử dụng phép trừ ảnh:**
 - **Phát hiện thay đổi:** Phép trừ ảnh thường được dùng để phát hiện những thay đổi nhỏ giữa hai hình ảnh liên tiếp, ví dụ như trong các hệ thống giám sát, theo dõi chuyển động.
 - **Phân tích hình ảnh:** Bằng cách trừ đi một hình ảnh nền, chúng ta có thể tập trung vào các đối tượng chuyển động hoặc các chi tiết quan trọng khác.
 - **Xử lý ảnh:** Phép trừ ảnh có thể được sử dụng như một bước tiền xử lý trong nhiều thuật toán xử lý ảnh khác.
- **Cách thức thực hiện phép trừ ảnh:**
 - **Kiểm tra kích thước:** Đảm bảo hai hình ảnh có **cùng kích thước**. Nếu không, cần thực hiện **resize** trước khi trừ.
 - **Trừ từng pixel:** Mỗi pixel trong hình ảnh kết quả sẽ bằng **giá trị pixel tương ứng trong hình ảnh A trừ đi giá trị pixel tương ứng trong hình ảnh B.**

$$g(x, y) = |f_1(x, y) - f_2(x, y)|$$
 - **Hiển thị kết quả:** Hình ảnh kết quả sẽ cho thấy những vùng sáng ở nơi có sự khác biệt lớn giữa hai hình ảnh gốc.



```
import cv2
import numpy as np

# Đọc hai hình ảnh
img1 = cv2.imread('image1.jpg')
img2 = cv2.imread('image2.jpg')

# Chuyển đổi hình ảnh thành grayscale
gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

# Thực hiện phép trừ
diff = cv2.absdiff(gray1, gray2)

# Hiển thị kết quả
cv2.imshow('Difference', diff)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

- **Chuyển sang ảnh xám (Grayscale Conversion)**

```
gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
```

- Để tìm sự khác biệt, ta thường chỉ quan tâm đến sự thay đổi về **cấu trúc/hình dạng** (độ sáng tối) chứ không quan tâm đến màu sắc (xanh đỏ tím vàng).
- Tuy nhiên, việc chuyển về ảnh xám (1 kênh màu) **giúp máy tính tính toán nhanh gấp 3 lần** so với để ảnh màu (3 kênh).
- **Vì thế ảnh KQ thường ở dạng grayscale.**

- $\text{diff} = \text{cv2.absdiff}(\text{gray1}, \text{gray2})$ tương đương $\text{Pixel}_{\text{moi}} = |\text{Pixel}_{\text{anh1}} - \text{Pixel}_{\text{anh2}}|$

- Nếu tại một điểm, 2 ảnh **giống hệt nhau**: $100 - 100 = 0$ (Màu đen).
- Nếu tại một điểm, 2 ảnh **khác nhau**: $|200 - 50| = 150$ (Màu xám sáng/Trắng).
- **Tại sao phải dùng trị tuyệt đối (abs)?** Để bắt được sự khác biệt bất kể ảnh nào sáng hơn. Nếu không có trị tuyệt đối, $50 - 200 = -150$, **máy tính sẽ coi số âm là 0 (đen)** và bạn sẽ bị mất thông tin.

- **Ứng dụng:**

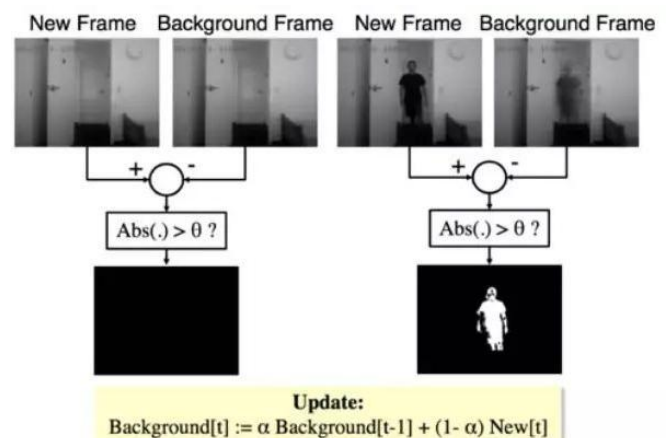
- Phát hiện chuyển động (Liveness Check)
- Phát hiện thay đổi cảnh (Scene Change Detection)
- Nhận diện khuôn mặt (Face Recognition) → Hoàn toàn đúng về mặt nguyên lý toán học cơ bản. Tuy nhiên, trong thực tế, Phép trừ ảnh trực tiếp trên pixel (Pixel-based Subtraction) là phương pháp **CỰC KỲ SƠ KHAI** và **HÀU NHƯ KHÔNG ĐƯỢC DÙNG** làm bước so sánh chính trong các hệ thống nhận diện khuôn mặt hiện đại.
 - Thay vì dùng Toán tử điểm (trừ pixel), hệ thống hiện đại dùng **Trí tuệ nhân tạo (Deep Learning)**.
 - **Vector hóa**: AI không nhìn pixel. Nó biến bức ảnh khuôn mặt thành một dãy số (ví dụ: 128 hoặc 512 con số), gọi là **Vector đặc trưng (Feature Vector)**. Dãy số này mã hóa khoảng cách 2 mắt, độ cao sống mũi, xương hàm, v.v. bất kể ánh sáng hay góc độ.
 - **So sánh Vector**: Khi so sánh ảnh lưu và ảnh quét, máy tính tính **khoảng cách toán học** giữa hai vector này (ví dụ: Khoảng cách Euclid hoặc Độ tương đồng Cosine).
 - **Đặt ngưỡng**: Nếu khoảng cách giữa 2 vector đủ nhỏ (dưới threshold) → Cùng một người.

3.8. Trừ nền trong video (Video Background Subtraction)

- Là một kỹ thuật **mở rộng từ phép trừ ảnh**, được áp dụng để phát hiện các đối tượng chuyển động trong một chuỗi các hình ảnh liên tiếp (video).
- Kỹ thuật này rất hữu ích trong các ứng dụng giám sát, theo dõi hành vi, và nhiều lĩnh vực khác.
- **Mục đích:**
 - *Phát hiện chuyển động*: Nhận diện các đối tượng đang di chuyển trong một cảnh tĩnh.
 - *Theo dõi đối tượng*: Đánh dấu và theo dõi các đối tượng đã được phát hiện.
 - *Phân tích hành vi*: Dựa trên các chuyển động để phân tích hành vi của các đối tượng.
- **Các phương pháp trừ nền phổ biến:**
 - **Trừ nền đơn giản**: Trừ từng frame của video với một hình ảnh nền tĩnh được chọn trước. Phương pháp này đơn giản nhưng dễ bị ảnh hưởng bởi các thay đổi về ánh sáng.
 - **Trừ nền thích ứng**: Tạo một mô hình nền động, liên tục cập nhật để thích ứng với các thay đổi về ánh sáng và các yếu tố môi trường khác.
- **Các thuật toán phổ biến:**
 - **ViBe**: Dựa trên các mẫu màu để mô hình hóa nền.
 - **KNN**: Sử dụng k-nearest neighbors để phân loại các pixel.
 - **MOG2**: Mô hình hỗn hợp Gaussian để mô hình hóa sự phân bố màu sắc của nền.
 - **Optical Flow**: Tính toán sự dịch chuyển của các pixel giữa hai frame liên tiếp để phát hiện chuyển động.
- **Thách thức trong việc trừ nền video**
 - **Ánh sáng thay đổi**: Ánh sáng thay đổi có thể ảnh hưởng đến hiệu quả của việc trừ nền.
 - **Bóng**: Bóng của các đối tượng chuyển động có thể gây nhiễu.
 - **Đối tượng mờ**: Các đối tượng chuyển động nhanh hoặc mờ có thể khó phát hiện.
 - **Nền phức tạp**: Nền có nhiều chi tiết hoặc thay đổi liên tục sẽ làm khó khăn cho việc phân biệt đối tượng và nền.
- **Ứng dụng của trừ nền video**
 - *Hệ thống giám sát*: Phát hiện xâm nhập, theo dõi phương tiện giao thông.
 - *Xe tự lái*: Phát hiện người đi bộ, xe cộ.
 - *Hệ thống tương tác*: Nhận diện cử chỉ, theo dõi mắt.
 - *Phân tích video*: Tính toán lưu lượng giao thông, phân tích hành vi người dùng.

- **Cơ chế hoạt động chi tiết của thuật toán "Trừ nền thích nghi" (Adaptive Background Subtraction).**

- **Bước 1: So sánh (Vòng tròn cộng trừ)**
 - Hệ thống lấy **"New Frame"** (Khung hình hiện tại từ Camera) trừ đi **"Background Frame"** (Hình ảnh nền mà máy tính đang ghi nhớ).
 - Lưu ý: "Background Frame" không cố định, nó được cập nhật liên tục theo thời gian (theo công thức Update).
- **Bước 2: Tính trị tuyệt đối và Phân ngưỡng ($Abs(.) > \theta$)**
 - **Abs(.)**: Lấy trị tuyệt đối của hiệu số (để không bị số âm). Đây chính là **cv2.absdiff**.



- Nếu sự khác biệt **nhỏ hơn ngưỡng θ** (ví dụ chỉ là nhiễu hạt camera, gió thổi nhẹ) → **Coi là 0** (Màu đen - Không có chuyển động).
- Nếu sự khác biệt **lớn hơn ngưỡng θ** (người xuất hiện) → **Coi là 1** (Màu trắng - Có chuyển động).
- **Bước 3: Kết quả → gọi là heatmap**
 - **Hình trái (Không có người):** Kết quả là màu đen tuyền.
 - **Hình phải (Có người):** Kết quả là bóng người màu trắng nổi trên nền đen. Đây chính là **fgmask** trong code.

○ **Công thức Trung bình trượt (Running Average) (Công thức Update):**

$$\text{Background}[t] := \alpha \cdot \text{Background}[t-1] + (1 - \alpha) \cdot \text{New}[t]$$

- **Background[t]:** Nền mới cập nhật.
- **Background[t-1]:** Nền cũ.
- **New[t]:** Ảnh hiện tại vừa quay được.
- **α (alpha):** Hệ số học (Learning Rate). Nó quyết định xem máy tính nên "bảo thủ" (giữ nền cũ) hay "tiếp thu" (cập nhật cái mới).
- **Ý nghĩa:**
 - Cái nền (Background) không phải là một bức ảnh chụp cố định từ đầu. Nó là **sự pha trộn giữa quá khứ và hiện tại**.
 - Nếu bạn mang một cái ghế vào phòng và để đó mãi: Lúc đầu máy sẽ báo đó là "vật thể lạ" (màu trắng).
 - Nhưng nhờ công thức này, sau một thời gian, cái ghế sẽ dần dần chìm vào trong "Background Frame" (trở thành một phần của nền). Máy sẽ không báo động nữa.

```
import cv2

# Tạo một đối tượng để trừ nền
fgbg = cv2.createBackgroundSubtractorMOG2()

# Đọc video
cap = cv2.VideoCapture('video.mp4')

while(True):
    ret, frame = cap.read()

    # Áp dụng thuật toán trừ nền
    fgmask = fgbg.apply(frame)

    # Hiển thị kết quả
    cv2.imshow('Frame', frame)
    cv2.imshow('FG Mask', fgmask)

    k = cv2.waitKey(30) & 0xff
    if k == 27:
        break

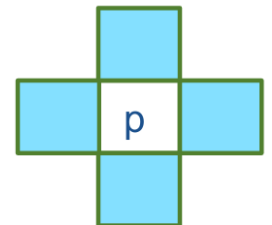
cap.release()
cv2.destroyAllWindows()
```

- Thuật toán **MOG2** được sử dụng để trừ nền. **Mỗi frame của video sẽ được so sánh với mô hình nền và tạo ra một mask** (mặt nạ) chỉ ra các vùng có đối tượng chuyển động (~ ảnh KQ).
- **fgbg = cv2.createBackgroundSubtractorMOG2():** □ Tạo ra một "bộ máy" trừ nền sử dụng thuật toán **MOG2** (Mixture of Gaussians).
 - Nếu chỉ dùng phép trừ đơn giản (absdiff), khi ánh sáng thay đổi hoặc có cảnh cây đung đưa, máy sẽ báo lỗi (coi đó là chuyển động).
 - **MOG2** có khả năng "**học**": Nó quan sát video theo thời gian để hiểu đâu là nền cố định, đâu là nhiễu (bóng cây, gợn nước) để bỏ qua. Nó tự động cập nhật mô hình nền liên tục.
- **k = ... break:** "Chiều xong một hình thì nghỉ 30ms nhé. Trong lúc nghỉ, nếu thấy tôi bấm nút **ESC** thì tắt chương trình ngay lập tức. Còn không thì cứ chiều tiếp đi."
- Trên một số máy tính (đặc biệt là 64-bit), hàm waitKey có thể trả về một dãy số dài loằng ngoằng. **Lệnh & 0xff** giúp cắt bỏ phần thừa, chỉ giữ lại **8 bit cuối cùng** (tương ứng với mã ASCII chuẩn của bàn phím). Giúp code chạy đúng trên mọi máy.

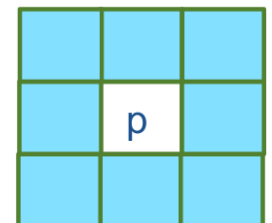
- **Vòng lặp xử lý từng khung hình (While Loop):**
 - **cap.read():** Đọc từng tấm ảnh (frame) từ video.
 - **ret (Return - Boolean):** Biến kiểm tra trạng thái.
 - Nếu đọc thành công (camera hoạt động tốt, video chưa hết): `ret = True`.
 - Nếu đọc thất bại (hết video, camera bị rút dây): `ret = False`. (Thường người ta sẽ thêm dòng `if not ret: break` để thoát vòng lặp).
 - **frame (Khung hình):** Chính là bức ảnh (dạng ma trận số) tại thời điểm đó. Đây là nguyên liệu đầu vào để chúng ta xử lý.
 - **fgmask = fgbg.apply(frame):** nơi thuật toán **Trừ nền thích nghi (MOG2)** hoạt động.
 - Nó lấy khung hình hiện tại, so sánh với cái "nền" mà nó đã học được trong quá khứ.
 - **Nhiệm vụ 1 (So sánh):**
 - Nó lấy frame hiện tại trừ đi "Mô hình nền" (Background Model) mà nó đang ghi nhớ trong đầu.
 - Kết quả trả về gán vào biến **fgmask** (Foreground Mask). Đây là bức ảnh đen trắng:
 - **Màu đen (0):** Là chỗ giống nền (tường, đất) (Background).
 - **Màu trắng (255):** Là vật thể đang chuyển động, chỗ khác biệt (Foreground) - ví dụ người đi bộ, xe chạy.
 - **Màu xám (có thể có):** Là bóng đổ (Shadow) của vật thể.
 - **Nhiệm vụ 2 (Học & Cập nhật - Update):**
 - Sau khi so sánh xong, nó âm thầm cập nhật lại "Mô hình nền" của chính nó.
 - Ví dụ: Nếu có một cái ghế mới được đặt vào khung hình và đứng yên, hàm `apply()` sẽ từ từ "học" cái ghế đó vào trong nền (theo công thức *Update* màu vàng trong ảnh sơ đồ bạn xem trước đó). Nhờ vậy, ở các khung hình sau, cái ghế sẽ không bị coi là vật thể lạ nữa.

4. Toán tử láng giềng

- Một khái niệm nền tảng trong xử lý ảnh, giúp chúng ta **khai thác thông tin từ các pixel lân cận** để thực hiện các phép biến đổi và phân tích hình ảnh.
- **Láng giềng của một pixel** là **tập hợp các pixel xung quanh pixel đó**.
- **Toán tử láng giềng** là một phép toán được **áp dụng lên một pixel và các láng giềng của nó để tính toán giá trị mới**.
- **Các loại láng giềng phổ biến**
 - **Láng giềng 4-kết nối:** Gồm các pixel ở trên, dưới, trái, phải của pixel đang xét.
 - **Láng giềng 8-kết nối:** Bao gồm cả các pixel ở các đường chéo.
 - **Láng giềng k-kết nối:** Là một khái niệm tổng quát, bao gồm các pixel nằm trong một vùng hình tròn có bán kính k xung quanh pixel đang xét.
- **Ứng dụng của toán tử láng giềng trong xử lý ảnh**
 - **Phát hiện cạnh.** Các cạnh trong ảnh thường có sự thay đổi đột ngột về cường độ sáng. Bằng cách **so sánh giá trị của một pixel với các láng giềng**, ta có thể phát hiện ra các cạnh. Các toán tử: **Sobel, Prewitt, Laplacian**.
 - **Làm mờ ảnh.** Có thể làm giảm nhiễu và làm mờ các chi tiết nhỏ trong ảnh bằng cách **tính trung bình giá trị của một pixel và các láng giềng của nó**. Các toán tử: **Trung bình, Gaussian blur**.
 - **Phân đoạn ảnh:** **nhóm các pixel có đặc trưng tương tự nhau** (ví dụ: màu sắc, cường độ) **thành các vùng**. Các toán tử: **Region growing, watershed**.

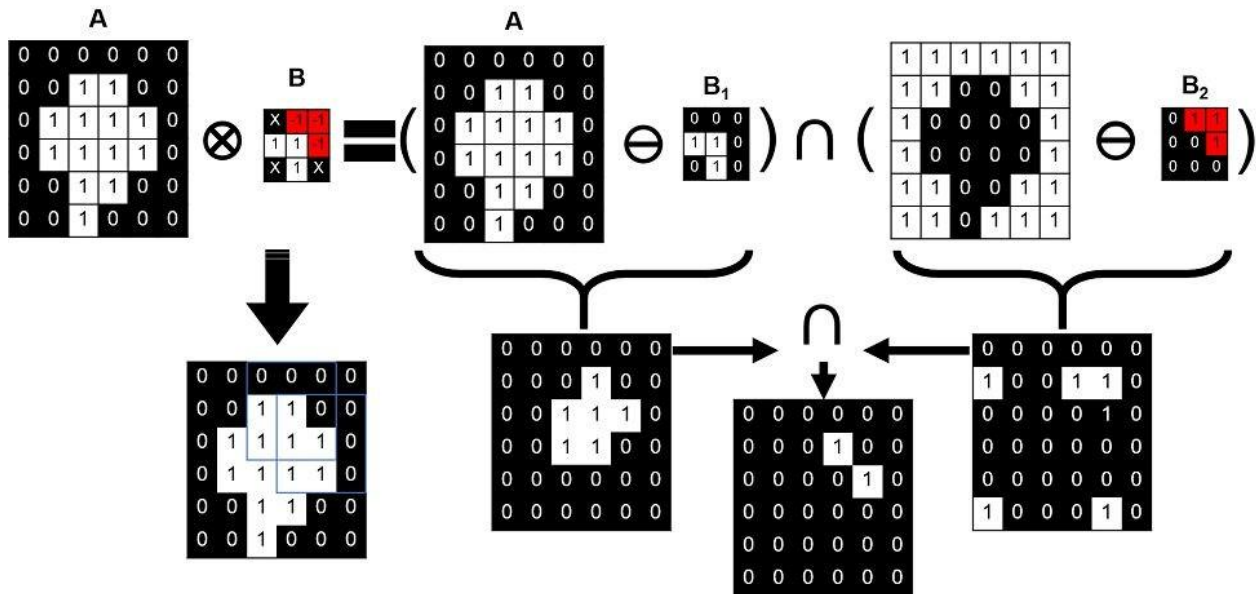


$N_4(p)$



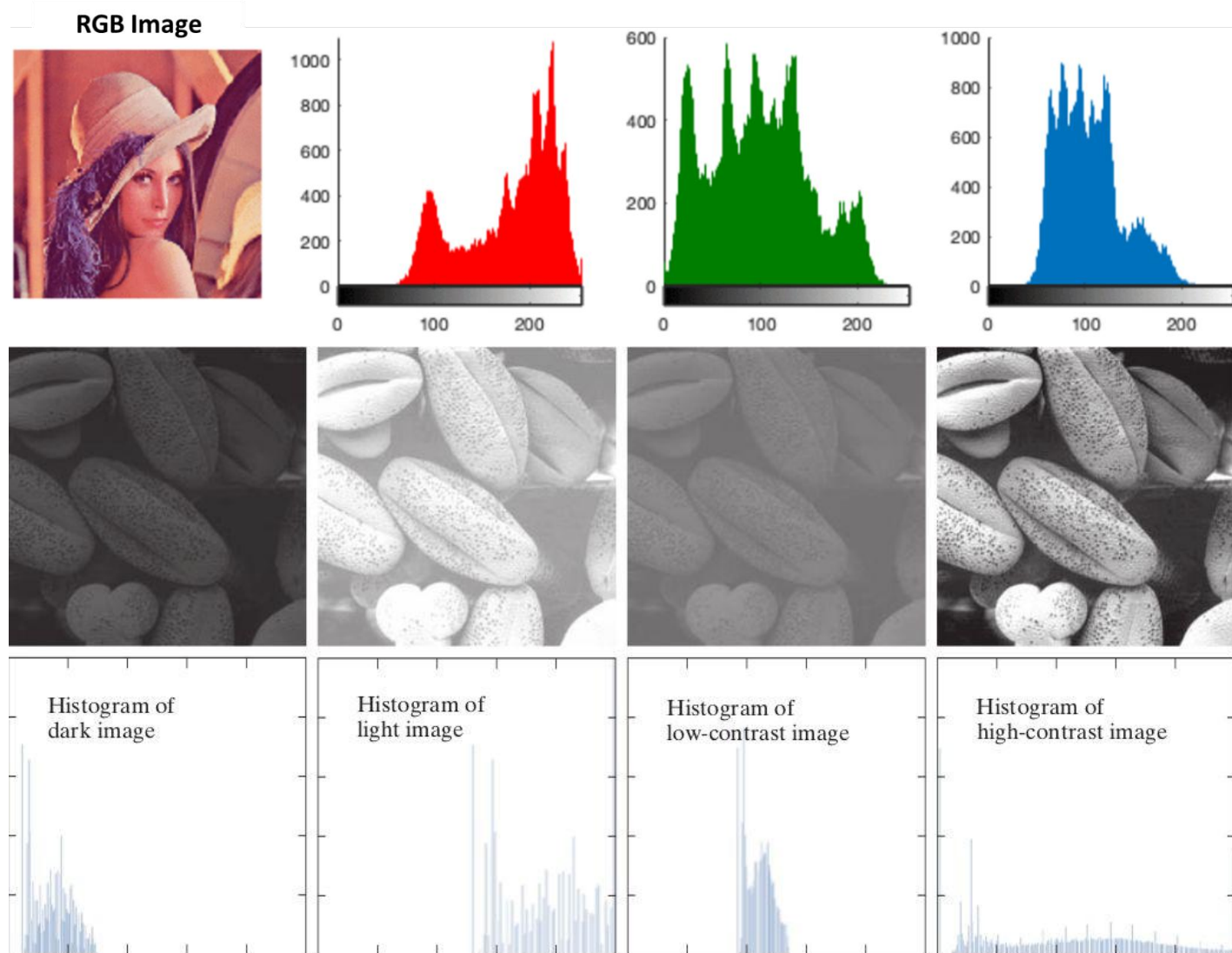
$N_8(p)$

- **Tăng cường ảnh:** **tăng cường các đặc trưng mong muốn của ảnh**, ví dụ như độ tương phản, độ sắc nét. Các toán tử: **Histogram equalization, unsharp masking**.
- **Phép biến đổi Hit-or-Miss (Hit-or-Miss Transform)** thuộc lĩnh vực Xử lý ảnh hình thái học (Mathematical Morphology), một ví dụ điển hình và chặt chẽ của việc sử dụng Toán tử láng giềng (Neighborhood Operator).

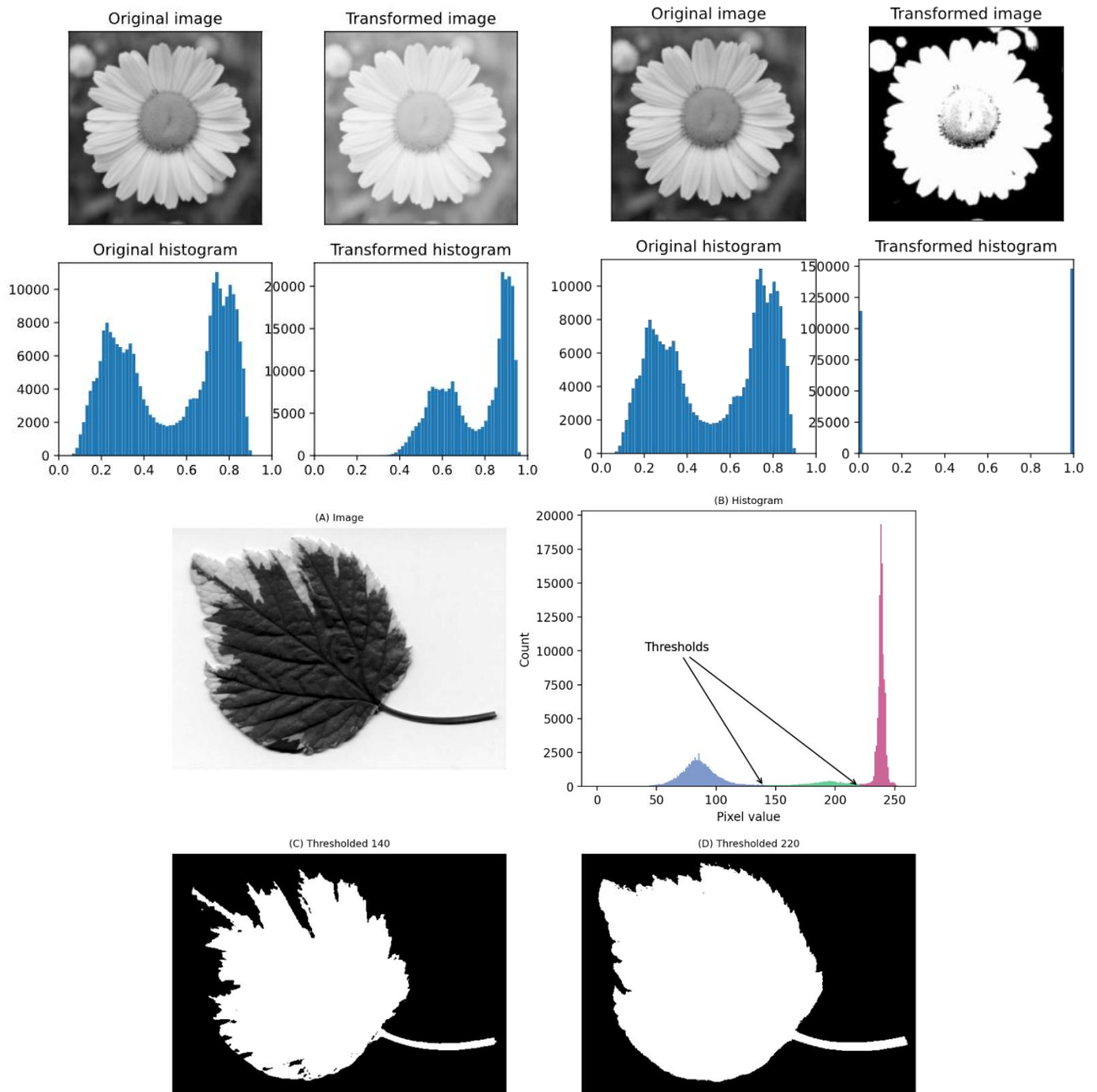


5. Histogram

- **Histogram** là một **biểu đồ thống kê** dùng để **biểu diễn phân phối tần suất** của dữ liệu **liên tục**.
- Trong xử lý ảnh, histogram cho ta biết **mỗi mức xám** (hoặc **mỗi màu** trong không gian màu khác) **xuất hiện bao nhiêu lần** trong một hình ảnh.
- Nói cách khác, histogram giúp chúng ta hiểu rõ về **sự phân bố các giá trị pixel** trong một hình ảnh.
- **Mục đích sử dụng Histogram:**
 - **Phân tích hình ảnh:** Histogram giúp chúng ta **đánh giá** độ tương phản, độ sáng tối, sự phân bố màu sắc của một hình ảnh.
 - **Cải thiện hình ảnh:** Nhiều kỹ thuật xử lý ảnh như **cân bằng histogram, tăng cường độ tương phản dựa trên thông tin từ histogram**.
 - **So sánh hình ảnh:** So sánh histogram của hai hình ảnh để **đánh giá mức độ tương đồng**.
- **Các loại Histogram:**
 - **Histogram mức xám:** Biểu diễn phân bố các mức xám trong một hình ảnh grayscale.
 - **Histogram màu:** Biểu diễn phân bố các màu trong một hình ảnh màu (ví dụ: RGB, HSV).
 - **Histogram cục bộ:** Tính histogram cho các vùng nhỏ trong hình ảnh.



- Với **ảnh dark** thì histogram có **các cột tập trung vào bên trái** tương ứng với màu tối
- Với **ảnh light** thì histogram có **các cột tập trung vào bên phải** chứa các pixel trắng
- Với **ảnh độ tương phản thấp** (low-contrast) thì histogram có **các cột tập trung ở giữa**
- Với **ảnh độ tương phản cao** (high-contrast) thì histogram **san đều** với các giá trị



- **Cách tính Histogram:**

- **Xác định số lượng bins:**
 - A **bin** represents a range of intensity values. (Bin - chứa số lần pixel đó xuất hiện).
 - Một **bin** là một khoảng giá trị.
 - Số lượng bins ảnh hưởng đến độ chi tiết của histogram.
- **Khởi tạo mảng đếm:** Tạo một mảng có kích thước bằng số lượng bins, tất cả các phần tử đều bằng 0.
- **Đếm số lượng pixel:** Duyệt qua từng pixel trong ảnh.
- **Xác định giá trị pixel thuộc vào bin nào.**

- **Tăng giá trị của *bin*** tương ứng lên 1.
- **Vẽ histogram:** Biểu diễn kết quả đếm dưới dạng một biểu đồ cột, trong đó trục hoành là giá trị pixel và trục tung là tần suất xuất hiện.

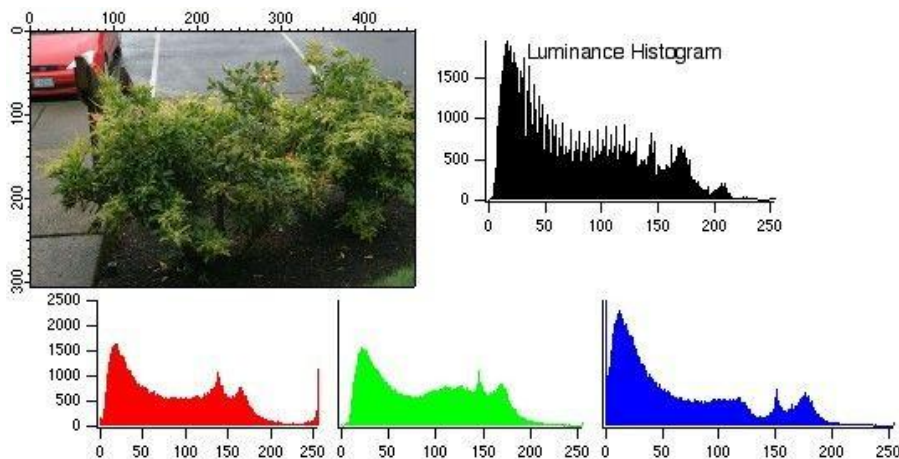
- **Xét 2 trường hợp (Small and Big bin)**

- **Small Bin Size (More Bins = More Detail)**

- Example: **256 bins for 256 grayscale values** (each bin represents 1 intensity level).
- If **bin size = 1**, each bin counts how many times a single pixel intensity appears.
- Example: **Bin 0** → 12 pixels with intensity 0, **Bin 255** → 9 pixels with intensity 255.
- More detailed histogram → Preserves fine variations in intensity (Tập trung những biến đổi nhỏ về cường độ).

- **Larger Bin Size (Fewer Bins = Less Detail)**

- Example: If **bin size = 3** (grouping 256 grayscale values into larger bins) => **Each bin now covers multiple intensity values.** (mỗi bin ở trường hợp này giữ 3 giá trị pixel)
- Example: **Bin 0 (covers 0–2)** → Counts **all pixels** with intensity 0, 1, or 2.
 - **Pixel 0** → 3 times
 - **Pixel 1** → 2 times
 - **Pixel 2** → 5 times
 - **Total count for Bin 0 = 3 + 2 + 5 = 10.**



Ví dụ bằng Python và OpenCV

- **cv2.calcHist** là hàm dùng để tính histogram.
- **[img]**: Danh sách các hình ảnh (ở đây chỉ có 1) (phải đặt trong ngoặc vuông).
- **[0]**: Kênh màu cần tính (Channel index). Chỉ định tính histogram cho kênh màu thứ 0 (grayscale).
- **None**: Không sử dụng mask → Tính toán cho toàn bộ bức ảnh (không che phần nào cả).
- **[256]**: HistSize/ Số lượng bins.
- **[0, 256]**: Dải giá trị của pixel muốn đo (từ 0 đến 255).
Dải [0, 256] được máy tính hiểu là: Lấy tất cả các giá trị v sao cho: $0 \leq v < 256$.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Đọc ảnh
img = cv2.imread('image.jpg', 0) # Đọc ảnh xám

# Tính histogram
hist = cv2.calcHist([img], [0], None, [256], [0, 256])

# Vẽ histogram
plt.plot(hist)
plt.xlabel('Intensity')
plt.ylabel('Number of pixels')
plt.show()
```

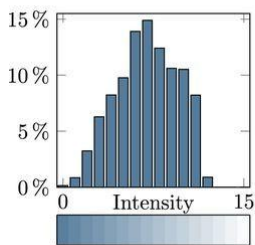
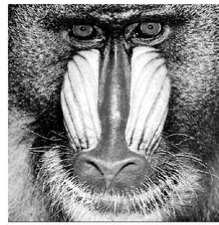
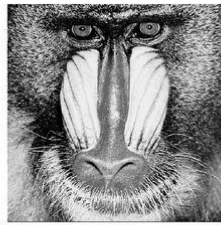
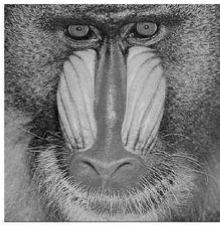
Thư viện Matplotlib để trực quan hóa mảng dữ liệu hist thành một đồ thị đường.

- Trục hoành (x): **Intensity** (Cường độ sáng của pixel, từ 0-255).
- Trục tung (y): **Number of pixels** (Số lượng pixel có cường độ sáng đó).

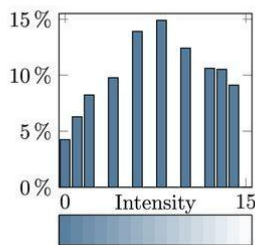
- **Histogram equalization (Cân bằng histogram)**

- **Tăng cường độ tương phản** của một hình ảnh bằng cách **phân phối lại các mức xám** trong histogram (tự động dàn đều histogram, thay vì focus ở một điểm). Điều này giúp cho hình ảnh trở nên *sáng hơn, rõ nét hơn và chi tiết hơn*, đặc biệt là trong các trường hợp hình ảnh có độ tương phản thấp hoặc các vùng sáng tối phân bố không đều.
- **Mục đích:**
 - *Tăng cường độ tương phản:* Làm cho các chi tiết trong hình ảnh trở nên rõ ràng hơn, đặc biệt là trong các vùng bóng tối hoặc vùng sáng quá sáng.
 - *Cải thiện chất lượng hình ảnh.*
 - *Chuẩn bị cho các bước xử lý tiếp theo*, cho các thuật toán xử lý ảnh khác như nhận dạng đối tượng, phân đoạn ảnh.
- **Ưu điểm:**
 - Đơn giản, dễ thực hiện.
 - Hiệu quả trong nhiều trường hợp.
 - Không yêu cầu thông tin về nội dung hình ảnh.
- **Nhược điểm:**
 - Có thể làm tăng nhiễu trong hình ảnh.
 - Không phù hợp với tất cả các loại hình ảnh, đặc biệt là các hình ảnh có nhiều chi tiết nhỏ.
- **Nguyên lý hoạt động**
 - **Tính toán histogram:** Tính histogram của hình ảnh gốc để biết phân bố các mức xám.
 - **Tạo hàm ánh xạ:** **Tạo một hàm ánh xạ mới**, trong đó các mức xám xuất hiện ít sẽ được ánh xạ sang các mức xám sáng hơn, và các mức xám xuất hiện nhiều sẽ được ánh xạ sang các mức xám tối hơn.

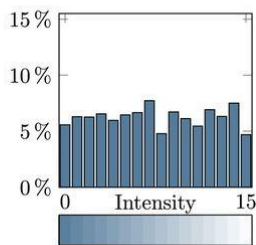
- **Áp dụng hàm ánh xạ:** Áp dụng hàm ánh xạ này **cho từng pixel** trong hình ảnh để tạo ra hình ảnh mới.
- **Công thức:** $s = T(r)$, trong đó:
 - **s:** Mức xám mới của pixel
 - **r:** Mức xám cũ của pixel
 - **T:** Hàm ánh xạ
- **Hàm ánh xạ thường được chọn là:** $T(r) = (L-1) \int_0^r p(x) dx$, trong đó:
 - **L:** Số lượng mức xám (thường là 256)
 - **p(x):** Xác suất xuất hiện của mức xám x



(a) Input



(b)



(c)

```
import cv2
import numpy as np

# Đọc ảnh
img = cv2.imread('image.jpg', 0)

# Cân bằng histogram
equ = cv2.equalizeHist(img)

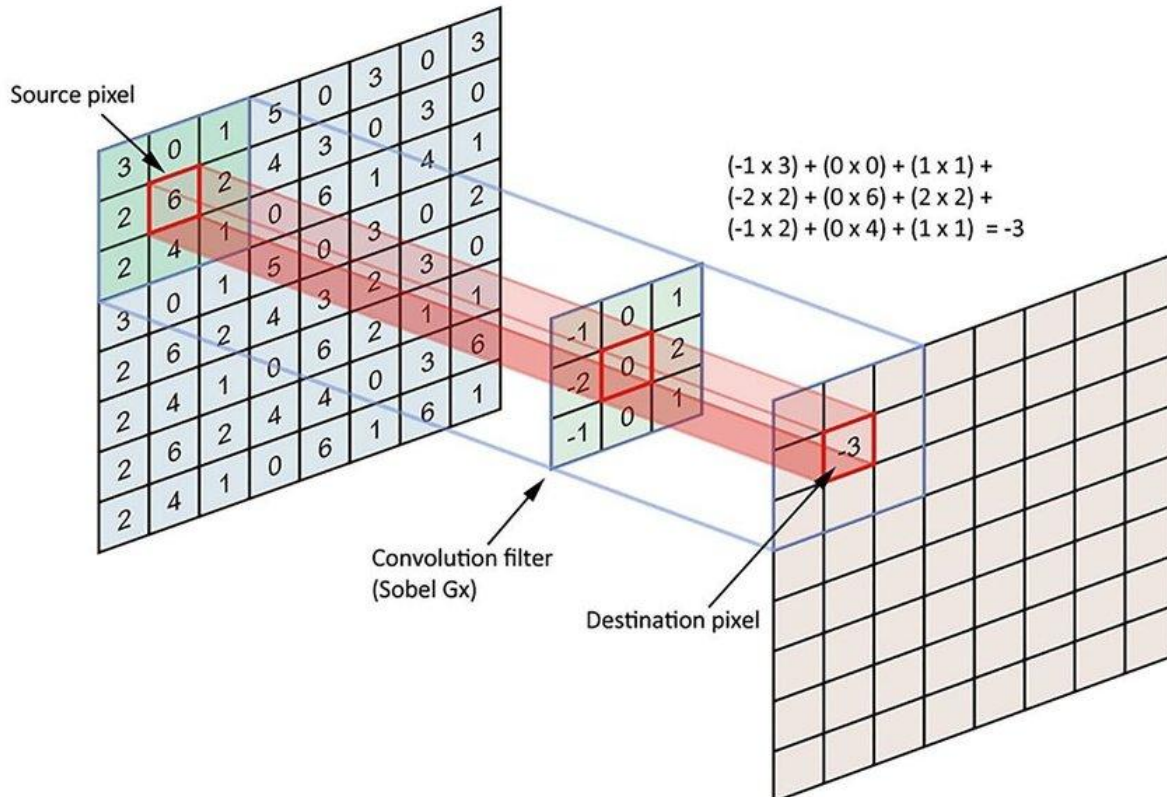
# Hiển thị kết quả
cv2.imshow('Original', img)
cv2.imshow('Equalized', equ)
cv2.waitKey(0)
```

6. Spatial Filtering - Các phép lọc trong không gian điểm ảnh

- **Spatial Filtering** (Lọc không gian) là một kỹ thuật cơ bản và quan trọng trong xử lý ảnh số.
- Nó hoạt động bằng cách **thay thế giá trị của mỗi pixel** trong ảnh **bằng một hàm tính toán dựa trên giá trị của pixel đó và các pixel lân cận**.
- Các **hàm tính toán** này thường được biểu diễn dưới dạng *một ma trận*, gọi là **kernel** hoặc **mask**.
- **Mục đích:**
 - **Loại bỏ nhiễu:** Loại bỏ các điểm ảnh nhiễu (chẳng hạn khi chụp trong tối, không đủ ánh sáng), làm cho hình ảnh mịn màng hơn.
 - **Làm mờ:** Làm mờ các cạnh và chi tiết nhỏ trong ảnh.
 - **Tăng cường cạnh:** Phát hiện và làm nổi bật các cạnh trong ảnh.
 - **Nhận dạng mẫu:** Tìm kiếm các mẫu nhất định trong ảnh.
- **Ví dụ về kernel lọc:**

Loại lọc	Kernel	Mô tả
Trung bình 3x3	[[1/9, 1/9, 1/9], [1/9, 1/9, 1/9], [1/9, 1/9, 1/9]]	Trung bình cộng các pixel trong vùng 3x3
Gaussian 3x3	[[1/16, 2/16, 1/16], [2/16, 4/16, 2/16], [1/16, 2/16, 1/16]]	Lọc Gaussian với sigma nhỏ
Laplacian 3x3	[[-1, -1, -1], [-1, 8, -1], [-1, -1, -1]]	Phát hiện cạnh

- **Mối liên hệ với CNN Kernel (Deep Learning):** Spatial Filtering CNN Kernel và cơ chế toán học là hoàn toàn giống nhau.
 - Trong Deep Learning (CNN - Convolutional Neural Networks), chúng ta cũng dùng chính phép tính **Tích chập (Convolution)** này.
 - Sự khác biệt duy nhất nằm ở **các con số bên trong Kernel**:
 - **Xử lý ảnh (Spatial Filtering): Số là hằng số** (Fixed), do con người chọn trước (như -1, 0, 1 trong ảnh).
 - **Deep Learning (CNN): Số là biến số** (Weights), khởi tạo ngẫu nhiên ban đầu và máy tính sẽ **tự học để thay đổi các số này** qua quá trình huấn luyện (Backpropagation) nhằm tìm ra các đặc trưng tốt nhất.



- **Sobel Gx** là một toán tử kinh điển trong xử lý ảnh truyền thống dùng để **phát hiện cạnh (Edge**

Detection) theo chiều dọc. Các con số trong ma trận (Kernel) là **cố định** $\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$. Các con

số này được các nhà toán học tìm ra và quy định sẵn để làm nổi bật sự thay đổi độ sáng từ trái sang phải.

- Trong **Spatial Filtering**, con người tự thiết kế ra các Kernel (như Sobel, Gaussian, Laplacian) để thực hiện một mục đích cụ thể (làm mờ, làm nét, tìm biên).
- **Ứng dụng của lọc không gian**:
 - *Xử lý ảnh*: Loại bỏ nhiễu, tăng cường cạnh, làm mờ, phát hiện đối tượng.
 - *Xử lý video*: Làm mịn video, ổn định video.
 - *Nhận dạng hình ảnh*: Trích xuất các đặc trưng hình ảnh.
 - *Xử lý ảnh y tế*: Tăng cường các chi tiết trong hình ảnh y tế.

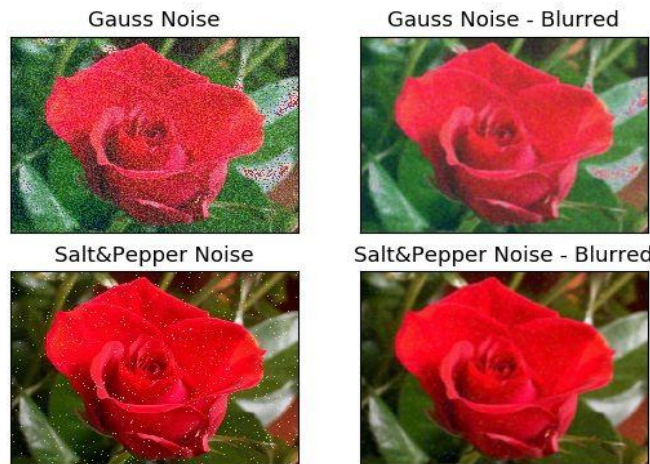
- **Các loại lọc không gian phổ biến:**

- **Lọc tuyến tính**

- *Lọc trung bình*: Mỗi pixel mới được tính bằng trung bình cộng của giá trị pixel đó và các pixel lân cận. Giúp làm mờ ảnh, loại bỏ nhiễu nhưng cũng làm mất chi tiết.
 - *Lọc Gaussian*: Sử dụng kernel Gaussian để làm mờ ảnh, các pixel gần tâm kernel có trọng số lớn hơn. Giữ lại các cạnh tốt hơn so với lọc trung bình.
 - *Lọc Laplacian*: Tính toán đạo hàm bậc hai của ảnh, giúp phát hiện các cạnh và chi tiết. Kernel Laplacian thường có các giá trị dương ở tâm và các giá trị âm xung quanh.

- **Lọc phi tuyến tính**

- *Lọc trung vị*: Thay thế mỗi pixel bằng trung vị của các giá trị pixel trong một vùng lân cận. Hiệu quả trong việc loại bỏ nhiễu muối tiêu.
 - *Lọc Min/Max*: Thay thế mỗi pixel bằng giá trị nhỏ nhất hoặc lớn nhất trong vùng lân cận. Sử dụng để phát hiện các đối tượng sáng hoặc tối.
 - *Lọc Bilateral*: Kết hợp cả thông tin không gian và cường độ để làm mờ ảnh. Giữ lại các cạnh trong khi làm mờ các vùng đồng nhất.



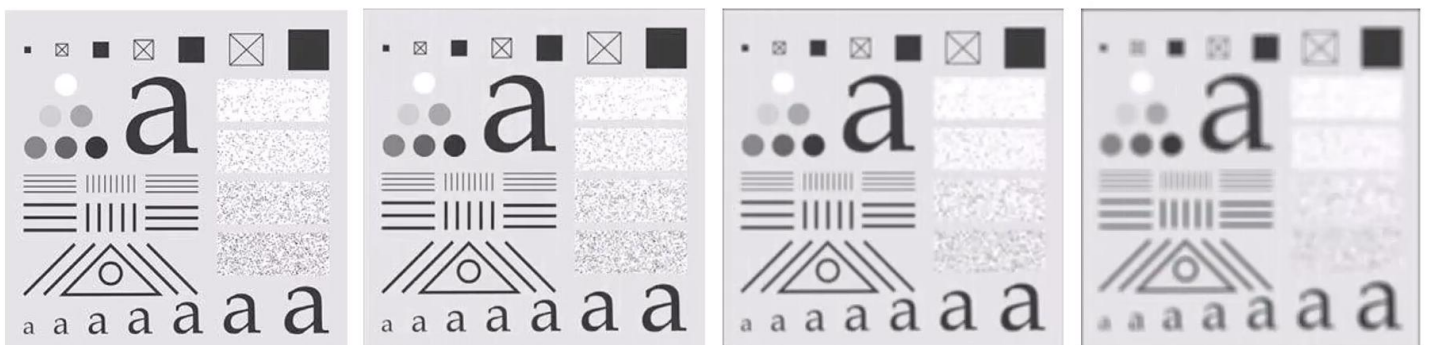
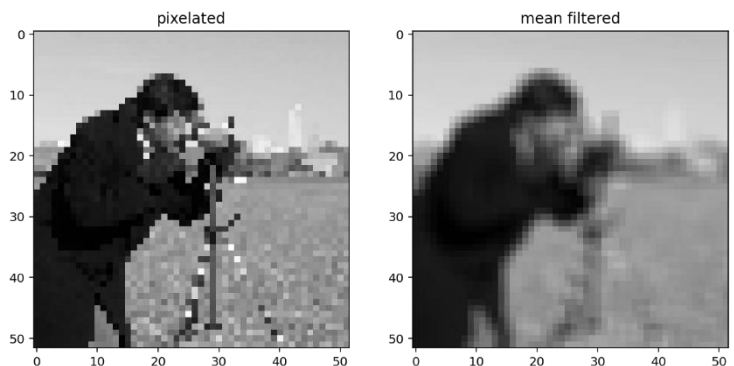
- **Lọc tuyến tính trong xử lý ảnh (linear processing filtering)**

- Nó hoạt động bằng cách thay thế giá trị của mỗi pixel trong ảnh bằng một tổ hợp tuyến tính của giá trị pixel đó và các pixel lân cận.
 - Tổ hợp tuyến tính này được xác định bởi một ma trận số gọi là **kernel hoặc mask**.
 - **Nguyên lý hoạt động**:
 - **Kernel**: Kernel là một ma trận nhỏ chứa các hệ số trọng số. Mỗi phần tử trong kernel tương ứng với một pixel trong vùng lân cận của pixel đang xét.
 - **Cuộn kernel**: Kernel được dịch chuyển qua từng pixel của ảnh.
 - **Tính toán giá trị pixel mới**: Tại mỗi vị trí, giá trị pixel mới được tính bằng cách nhân từng phần tử của kernel với giá trị pixel tương ứng trong vùng lân cận, sau đó cộng tất cả các kết quả lại.

- **Lọc trung bình:** Tất cả các phần tử trong kernel có giá trị bằng nhau → giúp làm mờ ảnh, loại bỏ nhiễu.
- **Lọc Gaussian:** Kernel có dạng phân phối Gaussian → Giữ lại các cạnh tốt hơn so với lọc trung bình.
- **Lọc Laplacian:** Kernel có các giá trị dương ở tâm và các giá trị âm xung quanh → Phát hiện các cạnh và chi tiết.
- **Ưu điểm và nhược điểm**
 - **Ưu điểm:** Đơn giản, dễ thực hiện; Hiệu quả cao; Có nhiều loại kernel khác nhau để lựa chọn.
 - **Nhược điểm:** Có thể làm mất một số chi tiết của ảnh, Không hiệu quả với các loại nhiễu phức tạp.

6.1. Lọc trung bình (Mean/Average Filter)

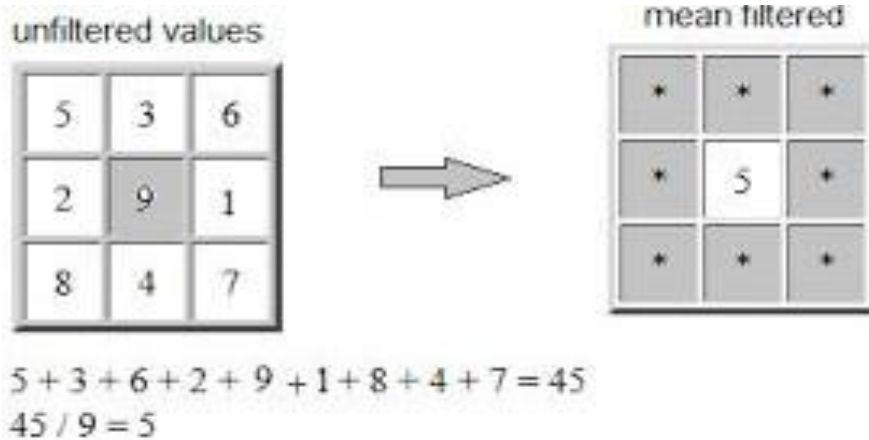
- Là một bộ lọc làm trơn (smooth filtering), được sử dụng để làm mịn hình ảnh và giảm nhiễu.
- Nguyên lý hoạt động của nó khá đơn giản:
 - **Mặt nạ lọc (cửa sổ):** Một ma trận số học nhỏ, thường là hình vuông (ví dụ: 3×3 , 5×5), được đặt lên từng điểm ảnh trong hình ảnh.
 - **Tính trung bình:** Giá trị tại điểm ảnh trung tâm của mặt nạ sẽ được thay thế bằng giá trị trung bình của tất cả các điểm ảnh nằm trong mặt nạ.
 - **Trượt mặt nạ:** Mặt nạ được di chuyển lần lượt qua từng điểm ảnh của hình ảnh để thực hiện quá trình tính trung bình.
- Ảnh gốc bên trái có kích thước 1024×1024 pixel
- Các ảnh tiếp theo biểu diễn ảnh lọc với lọc trung bình với kích thước 3×3 , 11×11 , 21×21 . Để ý thấy các chi tiết mất dần.



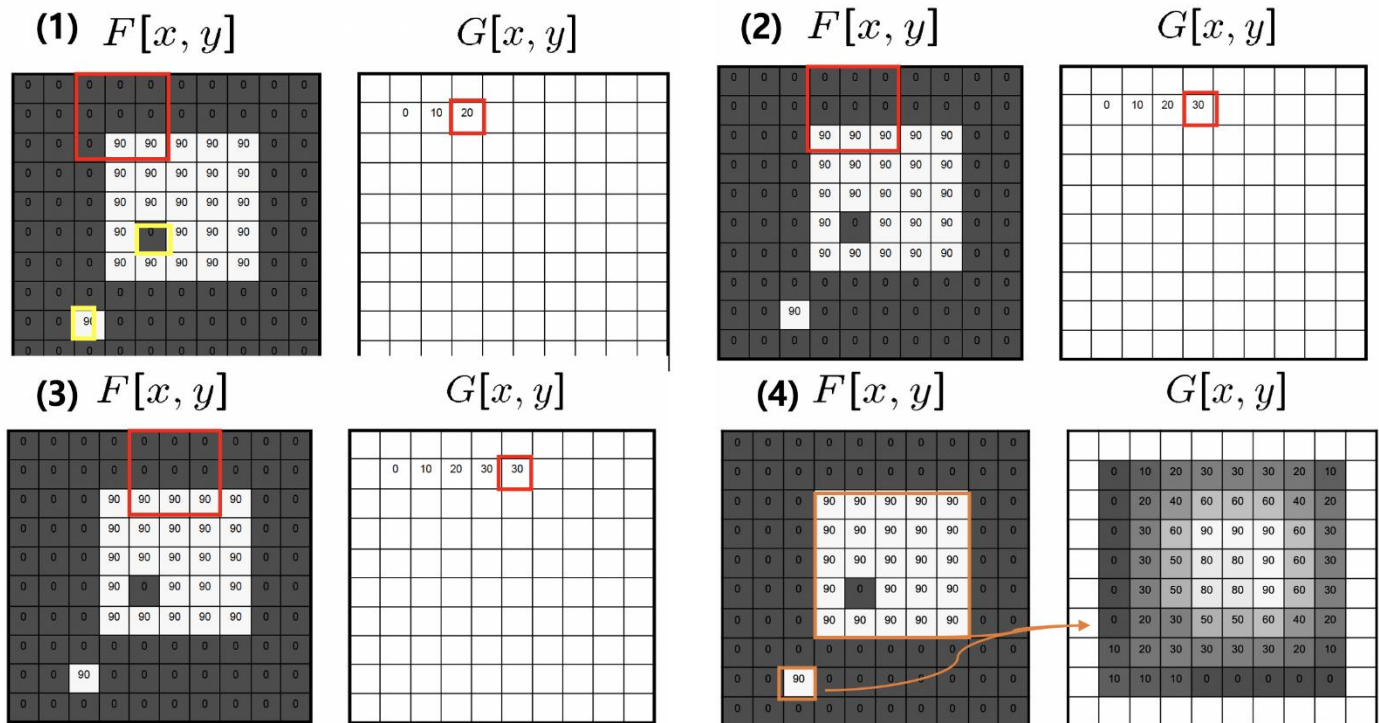
Ảnh gốc

Lọc kích thước 3×3 Lọc kích thước 11×11 Lọc kích thước 21×21

- **Ví dụ 1:** Giả sử ta có một mặt nạ lọc 3x3 và một phần hình ảnh như sau:



- **Ví dụ 2:** <https://www.youtube.com/watch?v=ZoaEDbivmOE>



- Khi cửa sổ trượt (kernel) 3x3 đặt tâm vào các pixel ở rìa mép ảnh, một phần của kernel sẽ bị "lòi" ra ngoài, nơi không có dữ liệu pixel nào cả. Để giải quyết việc "không có giá trị" này, trong thực tế người ta có 2 cách tiếp cận chính:
 - **Cách đơn giản: Bỏ qua (Valid Padding), chấp nhận ảnh kết quả bị nhỏ đi.**
 - Máy tính chỉ thực hiện tính toán ở những vị trí mà kernel nằm trọn vẹn bên trong ảnh.
 - **Hệ quả:** Nếu ảnh gốc kích thước $H \times W$, qua bộ lọc 3x3, ảnh mới sẽ bị mất đi 1 pixel ở mỗi cạnh $\rightarrow$ Kích thước còn $(H - 2) \times (W - 2)$.
 - Cách này **thường ít dùng** trong các ứng dụng cần giữ nguyên kích thước ảnh.

- **Cách phổ biến: Đệm thêm lề (Padding) - "Same Padding"**

- Để giữ cho ảnh đầu ra có kích thước **bằng y hệt** ảnh đầu vào (Input = Output), người ta sẽ tạo ra một viền giả xung quanh ảnh gốc *trước khi* thực hiện lọc. Kỹ thuật này gọi là **Padding**.
- Có 3 kiểu đệm lề phổ biến để "lấp đầy" chỗ trống khi kernel lòi ra ngoài:

- **Zero Padding (Đệm số 0):**

- Thêm một vòng các số 0 (màu đen) bao quanh ảnh.
 - *Ưu điểm:* Dễ tính toán.
 - *Nhược điểm:* Tự nhiên tạo ra một cái viền đen tối thui xung quanh ảnh, làm sai lệch giá trị trung bình ở mép.

- **Replicate Padding (Sao chép):**

- Pixel ở mép có giá trị bao nhiêu thì copy y nguyên ra ngoài lề.
 - *Ví dụ:* Hàng cuối là [10, 50, 90] thì lề thêm vào cũng là [10, 50, 90].

- **Reflect Padding (Phản chiếu - Hay dùng nhất):**

- *Coi mép ảnh là cái gương, phản chiếu hình ảnh bên trong ra ngoài.*
 - *Ưu điểm:* Giúp các chi tiết ở rìa ảnh trông tự nhiên, mượt mà, không bị khác như Zero padding.

- **Trong OpenCV (Code Python):** Khi bạn dùng hàm lọc, ví dụ cv2.blur (lọc trung bình) hoặc cv2.filter2D, bạn không cần lo lắng vì OpenCV **tự động xử lý Padding** cho bạn. Mặc định, OpenCV sử dụng kiểu **Reflect (Phản chiếu)**.

- Nếu bạn muốn kiểm soát nó, bạn có thể thêm tham số borderType:

```
# Lọc trung bình với kernel 3x3
# borderType=cv2.BORDER_CONSTANT nghĩa là dùng Zero Padding (viền đen)
result = cv2.blur(img, (3, 3), borderType=cv2.BORDER_CONSTANT)

# borderType=cv2.BORDER_REFLECT (Mặc định) nghĩa là dùng Reflect Padding
result_default = cv2.blur(img, (3, 3))
```

- **Kích thước của sổ bắt buộc phải là số LẺ (odd) như 3, 5, 7, 9... thay vì số chẵn (2, 4, 6...) do:**

- **Cần một "Tâm" xác định (Anchor Point)**

- Khi máy tính thực hiện bộ lọc (ví dụ như làm mờ), nó sẽ đặt cái cửa sổ Kernel trượt lên từng điểm ảnh của bức ảnh gốc.
 - **Quy tắc:** Kết quả tính toán của cả cái Kernel đó sẽ được gán cho **điểm ảnh nằm chính giữa Kernel**.

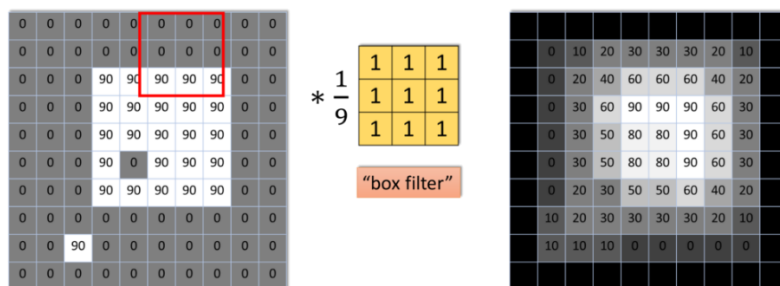
- Với Kernel Lẻ (ví dụ 3×3): $\begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \mathbf{X} & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix} \rightarrow$ Có một ô X nằm chính giữa. Máy tính biết ngay là: "À, tôi sẽ lấy các ô xung quanh để tính lại giá trị cho ô X này". Vị trí tâm là duy nhất và đối xứng.

- Với Kernel Chẵn (ví dụ 2×2): $\begin{bmatrix} \cdot & \cdot \\ \cdot & \cdot \end{bmatrix} \rightarrow$ Tâm nằm ở đâu? Tâm thực sự nằm ở giao điểm giữa 4 ô, chứ không rơi vào một ô pixel cụ thể nào cả. Máy tính sẽ không biết gán kết quả tính toán vào pixel nào.

○ Tránh bị lệch ảnh (Phase Shift)

- Nếu cố tình dùng Kernel chẵn (ví dụ 2×2) và ép máy tính chọn một ô làm tâm (ví dụ chọn ô góc trái trên), thì **toàn bộ bức ảnh sau khi lọc sẽ bị dịch chuyển (shift) sang một bên so với ảnh gốc**. Điều này làm sai lệch vị trí của vật thể trong ảnh (méo mó về mặt không gian).
- Kernel lẻ đảm bảo tính đối xứng hoàn hảo: Lề trái bằng lề phải, lề trên bằng lề dưới $\rightarrow$ Ảnh không bị xô lệch.

• Bản chất của lọc trung bình:

$$F(x, y) * H(u, v) = G(x, y)$$


$G = F * H$

```
import cv2
import numpy as np

# Đọc ảnh
img = cv2.imread('image.jpg', 0)

# Tạo kernel lọc
kernel = np.ones((5,5), np.float32)/25

# Áp dụng lọc
dst = cv2.filter2D(img,-1,kernel)

# Hiển thị kết quả
cv2.imshow('Original', img)
cv2.imshow('Filtered', dst)
cv2.waitKey(0)
```

- Máy tính sẽ thực hiện **phép tích chập (Convolution)** như ảnh trên, quy trình sẽ là:
 - Cộng:** Lấy 9 pixel trong vùng ảnh 3×3 nhân với ma trận 1 rồi cộng lại (Tức là tính **Tổng**).
 - Chia:** Nhân kết quả tổng đó với $\frac{1}{9}$ (Tức là **Chia cho số lượng phần tử**).

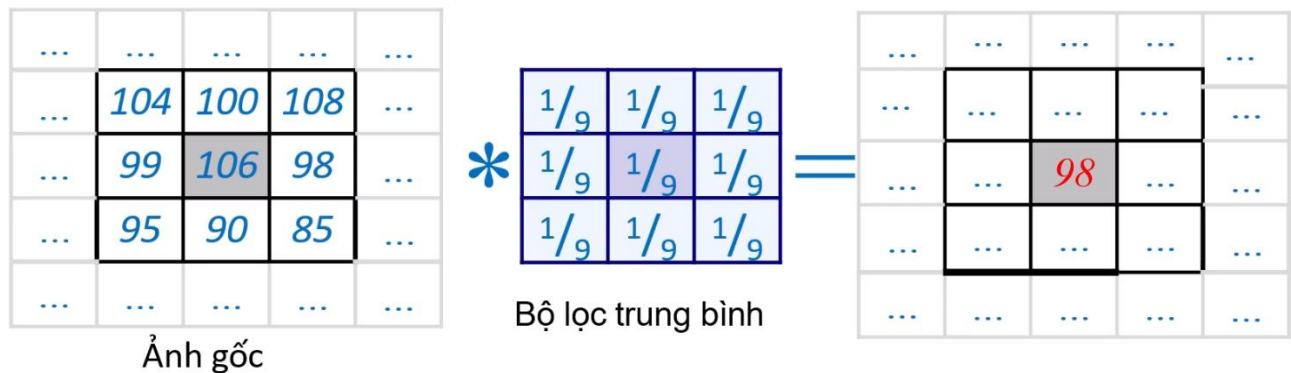
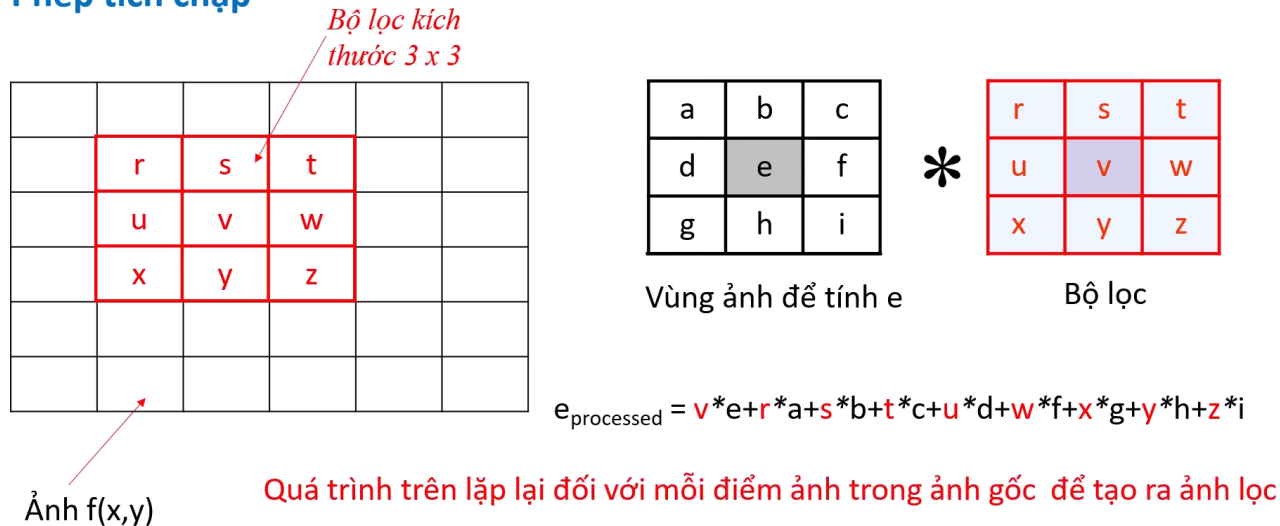
$\rightarrow$ **Tổng chia cho số lượng chính là Trung bình cộng.**
- Phía trên là code thực hiện **Lọc trung bình (Average Filtering)**, nhưng được viết theo cách "thủ công":
 - `kernel = np.ones((5,5), np.float32)/25` $\rightarrow$ tạo ra cái "cửa sổ trượt" (kernel) 5×5 :
 - `np.ones((5,5))`: Tạo một ma trận kích thước 5×5 toàn số 1.
 - `/25`: Chia toàn bộ ma trận cho 25. Khi cộng dồn giá trị của 25 pixel lại, ta phải chia cho 25 để lấy giá trị trung bình. Nếu không chia, giá trị pixel mới sẽ lớn gấp 25 lần ban đầu $\rightarrow$ Ảnh sẽ biến thành màu trắng xóa (cháy sáng). **Đây còn gọi là bước Chuẩn hóa (Normalize).**
 - `dst = cv2.filter2D(img, -1, kernel)` $\rightarrow$ Hàm này thực hiện phép Tích chập (Convolution) – chính là quy trình "trượt" kernel và nhân cộng.
 - img: Ảnh gốc.
 - 1 (depth): Tham số này bảo OpenCV **giữ nguyên độ sâu dữ liệu (bit depth) của ảnh đầu ra giống hệt ảnh đầu vào** (ví dụ: ảnh vào là uint8 thì ảnh ra cũng là uint8).
 - kernel: Cái màng lọc 5×5 chúng ta vừa tạo ở trên.
 - Trong thực tế, người ta ít khi tự tạo kernel rồi dùng filter2D như thế này (trừ khi muốn tùy chỉnh đặc biệt). OpenCV có sẵn hàm làm việc y hệt nhưng ngắn hơn:

```
# Cách chuyên nghiệp (kết quả y hệt code của bạn)
dst = cv2.blur(img, (5, 5))
```

Hoặc:

```
dst = cv2.boxFilter(img, -1, (5, 5))
```

Phép tích chập

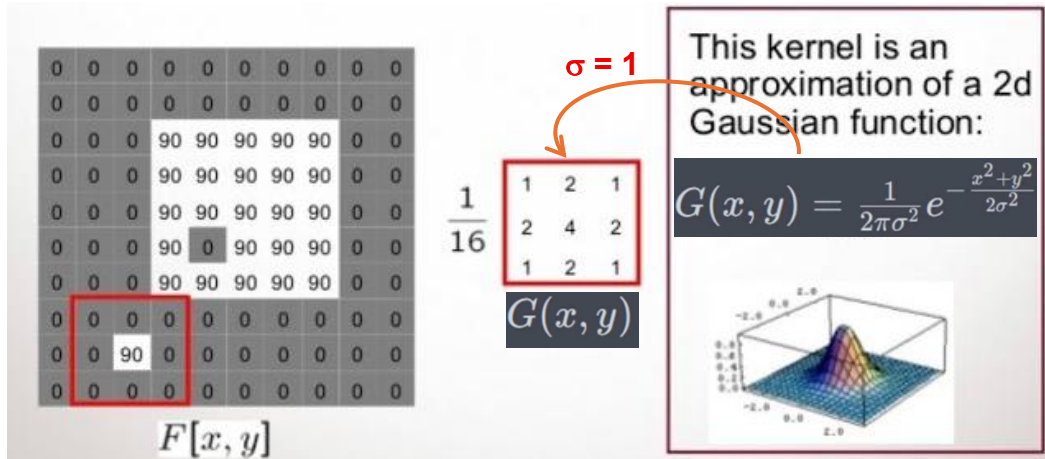


$$e = \frac{1}{9} \cdot 106 + \frac{1}{9} \cdot 104 + \frac{1}{9} \cdot 100 + \frac{1}{9} \cdot 108 + \frac{1}{9} \cdot 99 + \frac{1}{9} \cdot 98 + \frac{1}{9} \cdot 95 + \frac{1}{9} \cdot 90 + \frac{1}{9} \cdot 85 = 98.3333 \Rightarrow \text{Làm tròn} = 98$$

6.2. Lọc Gaussian

- Một kỹ thuật xử lý ảnh thường được sử dụng để **làm mịn hình ảnh, làm mờ/trơn ảnh và giảm nhiễu** một cách hiệu quả hơn so với lọc trung bình đơn giản.
- Khác với lọc trung bình, **lọc Gaussian sử dụng một ma trận trọng số (kernel) có các giá trị được xác định bởi phân phối Gaussian** (phân phối chuẩn), và được sử dụng để tạo ra hiệu ứng làm mờ trên ảnh bằng cách giảm độ biến đổi của màu sắc giữa các điểm ảnh gần nhau.
- Nguyên lý hoạt động**
 - Kernel Gaussian: là một ma trận vuông, **các giá trị trong ma trận này giảm dần theo khoảng cách tới tâm**. Các giá trị ở gần tâm có trọng số lớn hơn, tức là ảnh hưởng nhiều hơn đến giá trị điểm ảnh kết quả.
 - Quá trình lọc: Tương tự như lọc trung bình, kernel Gaussian được trượt qua từng điểm ảnh của hình ảnh. Tại mỗi vị trí, **giá trị của điểm ảnh trung tâm được tính bằng tổng trọng số của chính nó và các điểm ảnh lân cận nhân với giá trị tương ứng trong kernel**.

- **Công thức của mặt nạ lọc Gaussian** được tính dựa trên hàm Gaussian là (2D):



- $G(x, y)$ là giá trị tại điểm (x, y) của mặt nạ Gaussian.
- σ là tham số đặc trưng cho độ rộng của phân phối Gaussian (được gọi là độ lệch chuẩn).
- x và y là tọa độ của điểm ảnh trong mặt nạ.
- A simple 3x3 Gaussian kernel with standard deviation $\sigma = 1$: $\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$
 - The center pixel has the highest weight (4) because it's most important.
 - Nearby pixels have lower weights (2), and corner pixels have even smaller weights (1).
 - The sum of all values is 1 (ensuring no intensity loss).
- **1/16 → chuẩn hóa (normalize) kernel matrix**
→ Mục đích: (sum các giá trị trong kernel matrix)/ 16 = 1 → không có thay đổi brightness. Nếu bạn không chia cho 16, ảnh sau khi lọc sẽ có giá trị pixel sáng gấp 16 lần ảnh gốc (bị cháy sáng trắng xóa).
- **Có phải lúc nào cũng nhân với 1/16 ở Gaussian?**
 - **Câu trả lời: KHÔNG.** Con số 1/16 chỉ đúng trong trường hợp cụ thể là Kernel kích thước 3x3 với xấp xỉ số nguyên tiêu chuẩn. Hãy cộng tất cả các số trong ma trận này lại: $1+2+1+2+4+2+1+2+1=16$
 - **Vậy với các kích thước khác thì sao?** Nếu bạn dùng Kernel 5x5, các con số bên trong sẽ khác và tổng của chúng sẽ lớn hơn nhiều (ví dụ phổ biến là 273). Lúc đó, bạn sẽ phải nhân với $1/273 \rightarrow \text{Kernel}_{5 \times 5} \approx \frac{1}{273} \begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix}$
 - **Kết luận:** Con số để chia (1/16 hay 1/273...) **phụ thuộc vào tổng của tất cả các số trong ma trận Kernel** mà bạn đang dùng. Quy tắc bất di bất dịch là: Tổng các hệ số phải bằng 1 để giữ nguyên độ sáng trung bình của bức ảnh.



```
import cv2
import numpy as np

def gaussian_filter(img, kernel_size=5):
    return cv2.GaussianBlur(img, (kernel_size, kernel_size), 0)

# Đọc ảnh
img = cv2.imread('image.jpg')

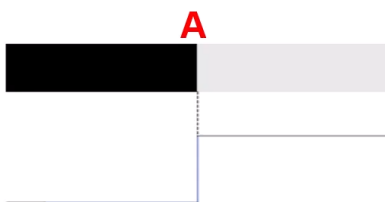
# Áp dụng lọc Gaussian
filtered_img = gaussian_filter(img, 5)

# Hiển thị ảnh
cv2.imshow('Original', img)
cv2.imshow('Filtered', filtered_img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

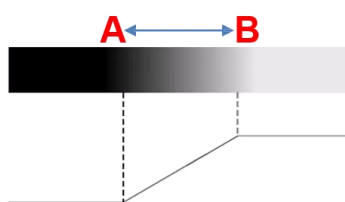
- `return cv2.GaussianBlur(img, (kernel_size, kernel_size), 0)`
 - **img**: Ảnh đầu vào.
 - **(kernel_size, kernel_size)**: Kích thước của Kernel (ví dụ 5×5), số này phải là **số lẻ**.
 - **0**: Đây là độ lệch chuẩn. Khi bạn để là **0**, OpenCV sẽ tự động tính toán con số này dựa trên kích thước kernel bạn nhập vào. Bạn không cần đầu đầu tính toán nó.

6.3. Lọc Laplacian

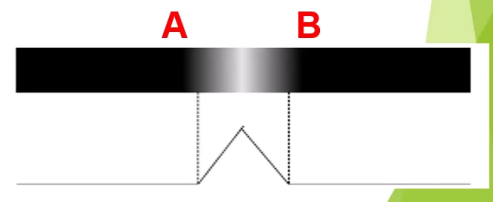
- **Lọc Laplacian** là một kỹ thuật xử lý ảnh được sử dụng để **phát hiện các cạnh/biên trong hình ảnh**. Nó dựa trên nguyên tắc rằng **các cạnh trong hình ảnh thường có sự thay đổi đột ngột về cường độ xám**.
- Là một loại lọc sắc nét ảnh trong miền không gian
- Một số khái niệm:
 - Biên ảnh (edge): là tập hợp các pixel mà tại đó giá trị mức xám (cường độ) thay đổi nhanh hoặc đột ngột trong một bức ảnh.
 - Mục đích của lọc sắc nét ảnh là làm nổi bật các chi tiết trong ảnh, tức là làm nổi các biên ảnh
 - Do vậy, lọc sắc nét cũng được gọi là kỹ thuật làm nổi biên ảnh → xác định sự thay đổi mức xám các pixel trong ảnh



biên nhảy vọt (skip)



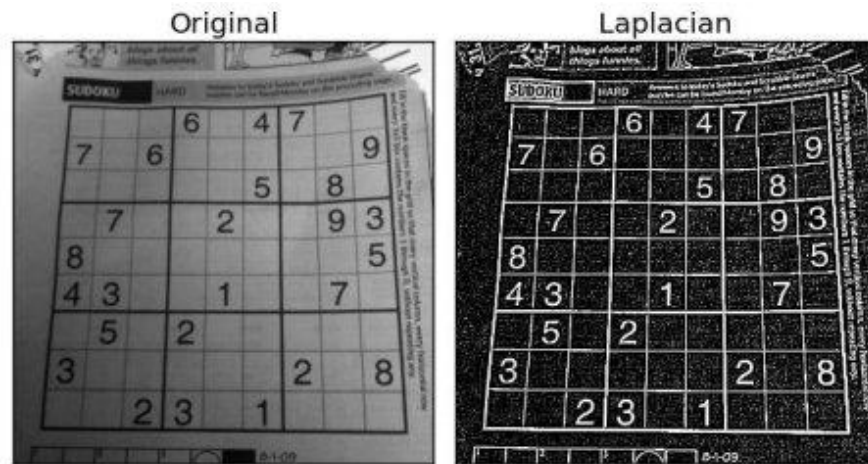
biên thoải thoải (Ramp)



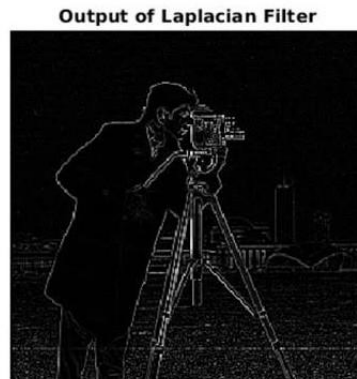
biên kiểu mái nhà (Roof)

- **Nguyên lý hoạt động**
 - **Kernel Laplacian**: Kernel Laplacian là một ma trận vuông, thường có kích thước 3×3 . Các giá trị trong ma trận này được thiết kế để **tính đạo hàm bậc hai của cường độ xám tại mỗi điểm ảnh**.

- **Quá trình lọc:** Kernel Laplacian được trượt qua từng điểm ảnh của hình ảnh. Tại mỗi vị trí, giá trị của điểm ảnh trung tâm được tính bằng tổng trọng số của chính nó & các điểm ảnh lân cận nhân với giá trị tương ứng trong kernel.
- **Phát hiện cạnh:** Các điểm ảnh có **giá trị lớn** trong kết quả lọc thường là các điểm nằm **trên cạnh** của hình ảnh.



- Tham khảo thêm về tính chất toán học để hình thành kernel lọc Laplacian:
<https://www.youtube.com/watch?v=WHx2cZ8MdGA>



```
import cv2
import numpy as np

def laplacian_filter(img):
    return cv2.Laplacian(img, cv2.CV_64F)

# Đọc ảnh
img = cv2.imread('image.jpg', 0)

# Áp dụng lọc Laplacian
filtered_img = laplacian_filter(img)

# Hiển thị ảnh
cv2.imshow('Original', img)
cv2.imshow('Filtered', filtered_img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

- `img = cv2.imread('image.jpg', 0)`: Tham số 0: Đọc ảnh dưới dạng **Ảnh xám (Grayscale)**. Kỹ thuật phát hiện cạnh (như Laplacian, Sobel, Canny) hầu như luôn luôn làm việc trên ảnh xám vì chúng quan tâm đến **cường độ sáng** thay vì màu sắc.
- `cv2.Laplacian(img, cv2.CV_64F)` → **Tại sao phải dùng cv2.CV_64F?**
 - **Toán học:** Laplacian tính toán dựa trên **Đạo hàm bậc 2** (**sự thay đổi của độ dốc**).
 - Khi chuyển từ Vùng sáng → Vùng tối: Độ dốc thay đổi theo chiều **Âm**.
 - Khi chuyển từ Vùng tối → Vùng sáng: Độ dốc thay đổi theo chiều **Dương**.
 - **Vấn đề:** Ảnh thông thường (uint8) chỉ chứa số từ 0 đến 255. Nó không hiểu số Âm. Nếu kết quả tính ra là -50, nó sẽ bị quy về 0 (đen) → Điều này làm mất một nửa số cạnh của vật thể.

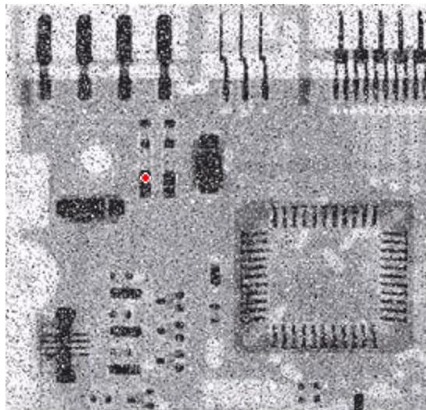
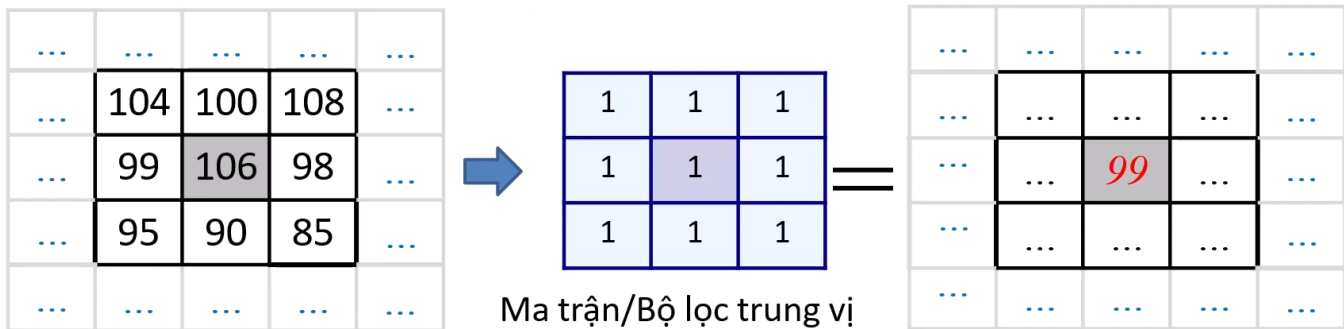
- **Giải pháp:** `cv2.CV_64F` là kiểu dữ liệu số thực (float) cho phép lưu trữ số âm. Dùng `CV_64F` giúp giữ lại các giá trị âm này để tính toán chính xác. Nếu bạn không dùng, bạn sẽ bị mất mát các cạnh chuyển từ Sáng sang Tối.
- Trong thực tế, để hiển thị ảnh Laplacian đẹp mắt cho mắt người xem (loại bỏ số âm), người ta thường thêm một bước chuyển đổi giá trị tuyệt đối:

`filtered_img = cv2.convertScaleAbs(filtered_img).`

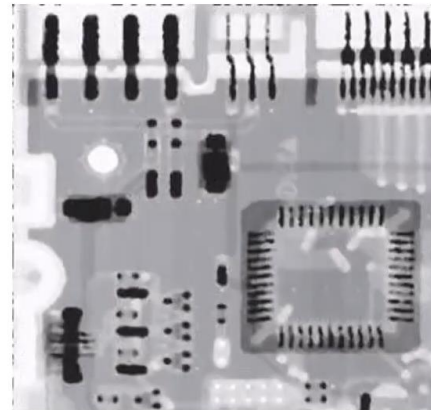
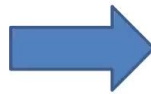
6.4. Một số kỹ thuật lọc khác

• Kỹ thuật lọc mịn ảnh – Lọc trung vị

- Giá trị trung vị X của một tập hợp là giá trị sao cho một nửa giá trị trong tập nhỏ hơn hoặc bằng X và một nửa còn lại lớn hơn hoặc bằng X . Thực hiện:
 - Sắp xếp giá trị của các pixel trong vùng lân cận (có kích thước bằng kích thước bộ lọc). Xác định giá trị trung vị của chúng và gán giá trị đó cho pixel trong ảnh được lọc



Hình ảnh chụp X-quang của một bảng mạch, bị hỏng do nhiễu hạt tiêu



Giảm nhiễu bằng cách sử dụng bộ lọc trung vị 7×7

- **Kỹ thuật lọc mịn ảnh – Lọc Min:** Xác định giá trị Min trong lân cận của pixel và gán giá trị đó cho pixel trong ảnh được lọc



- **Kỹ thuật lọc mịn ảnh – Lọc Max:** Xác định giá trị Max trong lân cận của pixel và gán giá trị đó cho pixel trong ảnh được lọc



7. Gradient and edge

- **Gradient.** Trong xử lý ảnh, gradient được hiểu đơn giản là độ dốc của cường độ pixel tại một điểm ảnh nhất định. Nó cho biết **mức độ thay đổi của cường độ sáng khi di chuyển từ điểm ảnh đó đến các điểm ảnh lân cận.**
- Gradient thường được **biểu diễn dưới dạng một vector, chỉ ra hướng và độ lớn** của sự thay đổi này.
- **Mục đích sử dụng Gradient:**
 - **Phát hiện cạnh:** Các cạnh trong một hình ảnh thường là nơi có sự thay đổi đột ngột về cường độ sáng. Do đó, bằng cách tính toán gradient, ta có thể xác định vị trí của các cạnh này.
 - **Phân tích hình ảnh:** Gradient cung cấp thông tin về cấu trúc và chi tiết của hình ảnh, từ đó hỗ trợ cho các nhiệm vụ như phân đoạn, nhận dạng vật thể, và theo dõi chuyển động.
- **Edge (biên/cạnh)** trong xử lý ảnh là **đường bao quanh các vùng có tính chất khác nhau trong hình ảnh.** Các cạnh thường **tương ứng với các thay đổi đột ngột về màu sắc, cường độ, hoặc kết cấu.**
- **Mối quan hệ giữa Gradient và Edge:**
 - **Gradient xác định Edge:** Các điểm có *giá trị gradient lớn thường nằm trên các cạnh* của hình ảnh.
 - **Edge là tập hợp các điểm gradient lớn:** Một cạnh có thể được định nghĩa là *một tập hợp các điểm liên kế có giá trị gradient lớn.*
- **Các toán tử tính gradient phổ biến:**
 - **Sobel:** Toán tử Sobel là một trong những toán tử phổ biến nhất để tính gradient. Nó **sử dụng hai kernel (kernel ngang và kernel dọc) để tính đạo hàm theo phương ngang và phương dọc.**

- **Prewitt:** Tương tự như Sobel, toán tử Prewitt cũng sử dụng hai kernel để tính gradient. Tuy nhiên, các hệ số trong kernel Prewitt đơn giản hơn so với Sobel.
- **Laplacian:** Toán tử Laplacian tính toán Laplacian của hình ảnh, tức là **tổng của đạo hàm bậc hai theo phương ngang và phương dọc**. Laplacian thường được sử dụng để phát hiện các cạnh và các điểm cực trị trong hình ảnh.
- **Ứng dụng của Gradient và Edge:**
 - **Phát hiện cạnh:** Đây là ứng dụng cơ bản nhất của gradient và edge. Các cạnh phát hiện được có thể được sử dụng để phân đoạn hình ảnh, trích xuất các đặc trưng hình dạng, hoặc theo dõi chuyển động.
 - **Phân đoạn hình ảnh:** Bằng cách sử dụng thông tin về gradient, ta có thể phân chia hình ảnh thành các vùng đồng nhất dựa trên các cạnh.
 - **Nhận dạng vật thể:** Các cạnh là các đặc trưng quan trọng để nhận dạng các vật thể trong hình ảnh.
 - **Theo dõi chuyển động:** Bằng cách so sánh các cạnh trong các khung hình liên tiếp, ta có thể theo dõi chuyển động của các vật thể.

8. Phụ lục

8.1. "Multimedia + Deep Learning" chắc chắn có sai số?

- Khác với lập trình truyền thống (Rule-based) nơi if A then B luôn đúng tuyệt đối, **Deep Learning là mô hình xác suất (Probabilistic Model)**.
- **Bản chất:** Mô hình học từ dữ liệu (Data-driven). Nó không "biết" sự thật, nó chỉ đưa ra dự đoán dựa trên những gì nó đã thấy.
- **Nguồn gốc sai số:**
 - **Dữ liệu (Data):** Ảnh bị nhiễu, mờ, thiếu sáng, hoặc nhãn dán (label) của con người ban đầu bị sai.
 - **Mô hình (Model):** Mô hình bị **Overfitting** (học vẹt, chỉ giỏi trên tập train) hoặc **Underfitting** (quá đơn giản, không học được đặc trưng).
 - **Độ phức tạp:** Thế giới thực vô cùng đa dạng (ví dụ: hàng triệu kiểu chữ viết tay), mô hình không thể bao quát 100% trường hợp.

→ Do đó, chúng ta không bao giờ khẳng định độ chính xác 100%, mà luôn phải chấp nhận một tỷ lệ sai số (Error rate) và tìm cách tối ưu các chỉ số đánh giá (Metrics).

- **Một vài chỉ số đánh giá: Precision, Recall và TPR**
 - Giả sử bài toán: Phát hiện khuôn mặt tội phạm (Positive = Có tội phạm, Negative = Người thường).
 - **TP (True Positive):** Có tội phạm, máy báo ĐÚNG là có. (Tốt)
 - **TN (True Negative):** Không có, máy báo ĐÚNG là không. (Tốt)
 - **FP (False Positive - Báo động giả):** Không có, máy NHẦM là có. (Sai)
 - **FN (False Negative - Bỏ sót):** Có tội phạm, máy NHẦM là không. (Sai - Nguy hiểm)
 - **Precision (Độ chính xác)**
 - **Câu hỏi:** "Trong số những tấm ảnh máy **BÁO LÀ CÓ**, bao nhiêu tấm đúng là có thật?"
 - Công thức: $Precision = \frac{TP}{TP + FP}$
 - **Ý nghĩa:** Đo lường độ tin cậy của dự đoán. Precision thấp nghĩa là máy hay "báo động giả" (báo nhầm người thường thành tội phạm).
 - **Recall (Độ phủ / Độ nhạy)**
 - **Câu hỏi:** "Trong số tất cả các tấm ảnh **CÓ THẬT** trong thực tế, máy tìm ra được bao nhiêu tấm?"
 - Công thức: $Recall = \frac{TP}{TP + FN}$
 - **Ý nghĩa:** Đo lường khả năng bao quát. Recall thấp nghĩa là máy hay "bỏ sót" đối tượng.

- **TPR (True Positive Rate)**
 - **Sự thật: TPR chính là Recall.** Hai thuật ngữ này là một, chỉ khác nhau về ngữ cảnh sử dụng (TPR thường dùng khi vẽ biểu đồ ROC).
 - **Công thức:** $TPR = Recall = \frac{TP}{TP + FN}$
- **Lưu ý quan trọng (Trade-off):** Thường thì Precision và Recall tỉ lệ nghịch.
 - Muốn không bỏ sót (Recall cao) → Phải chấp nhận báo nhầm nhiều (Precision thấp).
 - Muốn báo đâu trúng đó (Precision cao) → Phải chấp nhận bỏ qua những ca nghi ngờ (Recall thấp).
 - Để cân bằng, người ta dùng chỉ số **F1-Score**.

8.2. Cấu trúc ảnh: Foreground (Tiền cảnh) vs Background (Hậu cảnh)

- Trong bài toán thị giác máy tính, việc phân định rạch ròi 2 thành phần này là bước đầu tiên của mọi quá trình xử lý.
 - **Foreground (Object / ROI - Region of Interest)**
 - Là vùng chứa thông tin quan trọng mà ta muốn phân tích (con mèo, khuôn mặt, biển số xe...).
 - **Nhiệm vụ của AI:**
 - **Detection:** Vẽ khung chữ nhật (Bounding box) bao quanh Foreground.
 - **Segmentation:** Tô màu từng pixel thuộc về Foreground để tách nền chính xác.
 - **Background (Nền / Nhiều)**
 - Tất cả những phần còn lại không chứa thông tin quan trọng (bầu trời, mặt đường, cây cỏ phía sau...).
 - **Vai trò:**
 - Trong nhận diện: Background thường gây nhiễu, làm giảm độ chính xác.
 - Trong ngữ cảnh (Context): Đôi khi Background giúp nhận diện Foreground (ví dụ: thấy "biển nước" giúp tăng xác suất nhận diện vật thể là "chiếc thuyền").
- **Mối liên hệ với sai số**
 - **Sai số do Background:** Nếu Background quá phức tạp (rối rắm) hoặc có màu sắc giống hệt Foreground (Camouflage - Ngụy trang), chỉ số **FN (Bỏ sót)** sẽ tăng cao → **Recall giảm**.
 - **Sai số do nhầm lẫn:** Nếu Background có họa tiết giống vật thể, máy sẽ nhận nhầm đó là vật thể → **FP (Báo nhầm)** tăng → **Precision giảm**.

8.3. Bài toán FaceID: False Positive

- False Positive = Anh A nhưng nhận diện thành anh B.
- Trong giới chuyên môn, người ta chia làm 2 chỉ số sống còn:
 - **FAR (False Acceptance Rate - Tỷ lệ chấp nhận sai):**
 - Đây chính là **False Positive**.
 - **Tình huống:** Kẻ trộm (A) cầm điện thoại của bạn (B) và mở khóa thành công.
 - **Hậu quả:** Mất tiền, mất dữ liệu, lộ bí mật. → **Cực kỳ nguy hiểm**.
 - **Tiêu chuẩn:** Các hệ thống như FaceID của Apple phấn đấu đưa tỷ lệ này về mức **1 phần triệu** (1/1.000.000).
 - **FRR (False Rejection Rate - Tỷ lệ từ chối sai):**
 - Đây là **False Negative**.
 - **Tình huống:** Chính chủ (B) cầm điện thoại nhưng máy không nhận ra (do đeo khẩu trang, trời tối).
 - **Hậu quả:** Bạn phải nhập mật khẩu bằng tay. → **Phiên phức, khó chịu** (nhưng vẫn an toàn).
- **Kết luận:** Trong bảo mật, người ta thà "giết nhầm hơn bỏ sót". Thà bắt chính chủ nhập pass 10 lần (FRR cao) còn hơn để lọt kẻ gian 1 lần (FAR phải cực thấp).

8.4. Bài toán Phát hiện cạnh & kNN

- **Bước 1: Phát hiện cạnh (Edge Detection) → Đường bao (Contours)**
 - Dùng thuật toán Canny hoặc Sobel (thuật toán hình học) để tìm cạnh. Sau đó dùng thuật toán "Find Contours" để nối các cạnh thành vòng khép kín.
 - **Kết quả:** Bạn tách được vật thể ra khỏi nền (Background Subtraction).
- **Bước 2: Phân loại (Classification)**
 - **Trích xuất đặc trưng (Feature Extraction):** Sau khi cắt được cái object (từ bước 1), bạn nén nó lại thành một vector số (ví dụ: rút ra Histogram màu, hoặc HOG - hướng của các cạnh).
 - **Dùng kNN để Phân lớp (Classification):** dùng kNN để so sánh **vector đó** với dữ liệu mẫu. Ví dụ: Vector của vật thể vừa cắt ra gần với vector "Con chó" hơn hay "Con mèo" hơn?

Danh mục tài liệu sử dụng:

- **Tài liệu học phần:** TS. Nguyễn Thị Khánh Tiên, *Chương 2: Các thuật toán xử lý ảnh (Part 1), Slide Học phần Xử lý ảnh & Thị giác máy tính*, Viện Công Nghệ Thông Tin, Điện, Điện Tử, Trường Đại Học Giao Thông Vận Tải Tp.HCM.
- **Tài liệu tham khảo:**
 - Trần Lê Anh (3/2025), *Chapter 2 – Part 1: Point Operators & Linear Filters, Slide Image Processing and Computer Vision*, Viện Công Nghệ Thông Tin, Điện, Điện Tử, Trường Đại Học Giao Thông Vận Tải Tp.HCM.
 - Trần Đăng Nam, *Lecture Note Computer Vision*.
 - TS. Đỗ Năng Toàn, TS. Phạm Việt Bình (11/2007), *Chương 1: TỔNG QUAN VỀ XỬ LÝ ẢNH, Giáo Trình Môn Học Xử Lý Ảnh*, Khoa Công Nghệ Thông Tin, Đại Học Thái Nguyên
 - ThS. TRẦN VĂN HÙNG, *Chương 2. Các Kỹ Thuật Cơ Bản Trong Xử Lý Ảnh, Slide Thị Giác Máy Tính (Computer Vision)*, Trường Đại học Công nghiệp TP.HCM
 - TITV (10/2023), *Thị giác máy tính 20. Bộ lọc Gaussian làm mờ ảnh*, https://www.youtube.com/watch?v=tkogqlquDa_8
 - EPS Online Education (3/2025), *Lọc mịn ảnh trong miền không gian*, https://www.youtube.com/watch?v=Fs_EkLqlzMl
 - EPS Online Education (3/2025), *Lọc sắc nét ảnh trong miền không gian*, https://www.youtube.com/watch?v=Fs_EkLqlzMl
 - Các công cụ AI hỗ trợ.